

Grata Cascade: Pochopitelný průvodce

Populárně-naučná verze paperu v9

Robert Jarušek
Grata Luma s.r.o., Praha

Co je tato knížka

Toto je *populárně-naučná verze* formálního paperu *Grata Cascade: Hash-based One-Shot Key Agreement via Statistical Drift, as a Candidate for Hybrid Post-Quantum Key Establishment* (v9). Obsahuje stejné věcné informace — všechna tvrzení, empirická čísla, tabulky a závěry — ale v jazyce, kterému porozumí každý, kdo ví, co je hashovací funkce a šifra. Pro úplnou matematiku, důkazy a citace odkazujeme na hlavní paper. Cíl: aby protokol pochopil i amatér.

Problém narozenin popisuje statický stav. Grata Cascade na něm staví dynamický jednorázový systém. Rozšíření na continuous (CKA) variant je otevřené.

Obsah

1 Úvod: Hra o tři čísla	2
1.1 Hra na pochopení	2
1.2 Z hračky do kryptografie	3
1.3 Proč by nás měl zajímat třetí nezávislý předpoklad?	3
1.4 Co všechno je dokázané a změřené	4
1.5 Vizualizace: stejná hra, rozdílné výsledky	4
1.6 Co najdete v této knížce	5
2 Statická asymetrie: zobecněný problém narozenin	6
2.1 Birthday paradox jako rozcvička	6
2.2 Co se stane se třemi stranami	6
2.3 Poměr asymetrie: srdce této knížky	6
2.4 Volba parametrů: trojúhelník kompromisů	7
2.5 Proč je toto všechno <i>statické</i>	8
3 Čtyři designová rozhodnutí	8
3.1 Rozhodnutí 1: Hash jako slibovací zámek	8
3.2 Rozhodnutí 2: Veřejný drift přes náhodný vektor	9
3.3 Rozhodnutí 3: Aktualizační pravidlo (inspirace TPM)	10
3.4 Rozhodnutí 4: AES bez paddingu (žádné validační orákulum)	10
3.5 Kombinovaný systém: jak to spolu funguje	11
4 Protokol v akci	12
4.1 Parametry: co je třeba znát, co je vnitřní detail	12
4.2 Pět parametrických profilů	13
4.3 Co se přesně děje v jednom kole	13
4.4 Vizualizace: jedno kolo přehledně	14

4.5	Tři role hashů, tři různá kalibrační omezení	14
4.6	Model útočníka	15
5	Statistický drift	15
5.1	Stat: veřejně rekonstruovatelná mapa	16
5.2	Inverzně-pravděpodobnostní výběr	17
5.3	Operační rozsah $p_{\text{upd}} \in [0,6, 0,95]$	17
5.4	Vizualizace: vývoj Stat distribuce přes několik kol	18
5.5	Proč je toto strukturálně jiné než TPM	18
6	AES bez paddingu jako obrana proti zaplavení	18
6.1	Per-round AES klíč	19
6.2	NoPadding: žádné validační orákulum	19
6.3	Útočník s velkým poolem: zaplavení místo identifikace	19
6.4	Proč krátký h_{BA} pomáhá obraně, ne útočnickovi	20
6.5	Cumulative preimage barrier a 2^{136}	20
6.6	Tři obrany dohromady	21
7	Empirické důkazy	21
7.1	Konvergence: 50 000 běhů, 4 divergence	22
7.2	Ablace $h_{AB} = 6$: jak posílit h_{AB} na nulové divergence	22
7.3	NIST SP 800-22: vypadají klíče jako náhodný šum?	23
7.4	Kalibrační sweepy: kde to přestává fungovat	23
7.5	Per-profile validace: funguje to i mimo referenci?	24
7.6	Pět tříd pasivních útočníků	24
7.7	Vizualizace: efektivnost útočníků vs. random baseline	25
7.8	Co celé experimenty říkají dohromady	25
8	Srovnání s jednorázovými KE primitivami	26
8.1	Hodnota: třetí nezávislá vrstva v hybridní kompozici	27
8.2	Vůči Merklovým puzzlům	27
8.3	Co Grata Cascade postrádá	28
9	Co ještě nevíme: otevřené problémy	28
9.1	Centrální problémy	28
9.2	Empirické rozšíření	29
9.3	Nové třídy útočníků	29
9.4	Inženýrská optimalizace	30
9.5	Výzkumné parametrické rodiny	30
9.6	Co je naopak <i>uzavřeno</i>	31
10	Glosář a kde dál	31
10.1	Slovník pojmů	31
10.2	Kde dál	33

1 Úvod: Hra o tři čísla

1.1 Hra na pochopení

Představte si jednoduchou hru. Tři lidé — Alice, Bob a odposlouchávačka Eva — sedí v oddělených místnostech. Každý dostane stejnou desku s čísly 1 až 10 a nezávisle si vybere tři z nich.

Alice a Bob se chtějí veřejně (přes Evou odposlouchávaný kanál) shodnout na nějakém čísle, které si *oba dva nezávisle vybrali*. Eva chce jejich shodu uhodnout.

Položme si dvě otázky.

Otázka 1. *Jaká je pravděpodobnost, že si Alice a Bob vybrali alespoň jedno společné číslo?* Vyplyvá to z kombinatoriky: ze 10 čísel si Bob vybírá 3, a šance, že žádné z nich není v Alicině trojici, je $\binom{7}{3}/\binom{10}{3} = 35/120 \approx 29\%$. Pravděpodobnost neprázdného průniku je tedy přibližně **71 %**. Slušně velká šance — v naprosté většině případů Alice s Bobem mají co sdílet.

Otázka 2. *Eva také vybírá tři čísla. Jaká je pravděpodobnost, že v Eviných číslech je některé z těch, na kterých se Alice a Bob shodli?* Aby Eva uspěla, musí být některé z jejích tří čísel současně v Alicině i Bobově trojici (tedy v $A \cap B$). To je klasický *trojstranný problém narozenin* a jeho řešení dává přibližně **26 %**.

Vidíte ten rozdíl? Alice s Bobem se shodují v 71 % případů. Eva, ač má naprosto stejné "výpočetní zdroje" (také vybírá tři čísla z deseti), trefí jejich shodu jen v 26 %. Poměr $71/26 \approx 2,7$. Eva je strukturálně v nevýhodě, přestože hraje úplně stejnou hru.

Strukturální asymetrie

V tomto *ilustrativním příkladu* (kde Eva nezná nic o Alicině ani Bobově výběru a hraje stejnou „vyber-tři“ partii) je její nevýhoda kombinatorická, ne výpočetní. Dvoustranný průnik je v průměru větší než třístranný a víc výpočtu Evě v této zjednodušené verzi nepomůže. Skutečný protokol Grata Cascade tuto intuici dále rozšiřuje, ale jeho bezpečnostní záruky pak stojí na kombinaci strukturálních argumentů a empirické evaluace — ne na prostém zobecnění této kombinatorické nerovnosti. K tomu se vrátíme v Kapitole 7.

V naší ukázce s deseti čísly je rozdíl jen $2,7\times$. V reálné kryptografii budeme mít prostor velikosti 2^{256} místo deseti a každý vybere $k = 4096$ vektorů místo tří. Poměr asymetrie pak nebude $2,7\times$, ale $\approx 2^{244}$ — astronomický. Detailně si to rozebereme v Kapitole 2.

1.2 Z hračky do kryptografie

Hra s deseti čísly je samozřejmě jen ukázka pro představu. Skutečná kryptografie pracuje s prostorem velikosti 2^{256} (vektory dlouhé 32 bytů, tedy $256^{32} = 2^{256}$ možných hodnot — číslo se 78 desítkovými ciframi) místo deseti. Ale princip zůstává: pokud najdeme způsob, jak Alice a Bob mohou *komunikovat veřejně* — a přitom z těch veřejných zpráv *nevyplyne*, jaký prvek průniku našli — získáváme stavební kámen pro kryptografický protokol.

To není triviální, protože komunikace musí splnit dvě věci současně:

1. Alice a Bob *musí* dokázat na něčem se shodnout (jinak je hra zbytečná).
2. Z veřejné komunikace *nesmí* být patrné, na čem se shodli (jinak Eva vyhraje).

Grata Cascade je odpovědí: parametrická rodina protokolů, která tohle zvládne. Není to první takový pokus — existují protokoly založené na Diffieho-Hellmanovi (klasické), na mřížích (postkvantové ML-KEM), na hashích v Tree Parity Machine (historické). Ale Grata Cascade má jednu vlastnost, která ji odlišuje: stojí pouze na *odolnosti hashů proti preimage* a na *statistickém driftu sdíleného stavu*. Žádný Diffie-Hellman. Žádné mříže. Jen hashe a sdílená informace.

1.3 Proč by nás měl zajímat třetí nezávislý předpoklad?

Vsuvka pro orientaci. V kryptografické literatuře se hodně mluví o *ratchetech* (čti [ráčet]) — mechanismech, které postupně mění klíč dopředu, nikdy zpět. To je užitečné pro *průběžnou* domluvu klíče v messagingu (Signal Triple Ratchet rotuje klíč mezi zprávami). *Tato práce ale*

ratchet nestaví — staví menší primitivu: jednorázovou domluvu klíče. To znamená "Alice a Bob spustí protokol, doběhne, dostanou jeden klíč, hotovo". Žádná průběžná rotace mezi zprávami. Rozšíření na ratchet je samostatný, otevřený směr (viz Kapitola 9, `prob:cka-extension`). Tady ho zmiňujeme jen pro orientaci, kde se Grata Cascade nachází.

Když si dnes (rok 2026) napíšete zprávu přes Signal nebo WhatsApp, vaše počáteční domluva klíče (než se rozjede ratchet) typicky používá *hybridní KEM*: kombinaci dvou jednorázových primitiv kombinujících klasickou a postkvantovou kryptografií. Konkrétně NIST doporučuje schéma zvané **X-Wing**, které kombinuje:

- **X25519** (eliptický Diffie–Hellman) — klasická vrstva. Zranitelná Shorovým algoritmem na kvantovém počítači.
- **ML-KEM-768** (mřížový KEM, dříve známý jako Kyber) — postkvantová vrstva. Stojí na obtížnosti Module-LWE problému.

Útočník musí prolomit *obě* vrstvy, aby získal klíč. To je *hybridní bezpečnost* — klasická crypto-praxe pro postkvantovou éru.

Otevřená otázka, kterou Grata Cascade adresuje, zní: *existuje třetí, prakticky použitelná jednorázová primitiva, která by stála na úplně jiném principu než Diffie–Hellman i mříže?* Pokud ano, hybridní stack by mohl mít tři nezávislé vrstvy — útočník by musel současně prolomit DLP, mříže i tu třetí asumci, což je strukturálně robustnější situace než dva pilíře.

Grata Cascade je kandidát na tu třetí vrstvu. Nestaví na DH ani na mřížích; staví výhradně na *odolnosti hashovací funkce vůči preimage* (předpokladu úplně jiného typu) v kombinaci se statistickým driftem. Tomu se v knížce postupně budeme věnovat.

1.4 Co všechno je dokázané a změřené

Než se pustíme do detailů, jedno věrné shrnutí: *protokol v testech konverguje téměř vždy, jeho výstupní klíče prošly standardní statistickou baterií jako náhodný šum, a ani jeden z pěti klasifikovaných pasivních útočníků nedostal v evaluovaném rozsahu klíč.*

Konkrétní čísla, intervaly spolehlivosti, kalibrační skeny a popis útočníků najdete v Kapitole 7 (Empirické důkazy). Tady jen pojmenujme rámec, abychom dál věděli, o čem se vlastně bavíme:

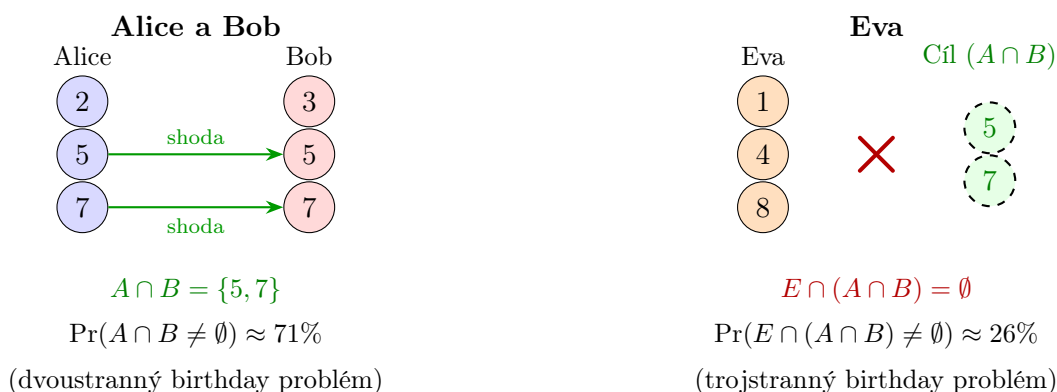
- Co je **změřeno empiricky**: konvergence protokolu, statistická uniformita výstupních klíčů, neúspěch konkrétních útočnických strategií.
- Co je **strukturálně argumentováno**, ale ne formálně dokázáno: bezpečnost vůči "TPM-stylovému" gradientnímu úniku, zachování statické asymetrie přes mnoho kol.
- Co **zůstává otevřené**: formální redukce dynamické asymetrie na obtížnost SHA-256 preimage, evaluace ML útočníků, agregace slabých signálů přes mnoho kol.

Tento rozdíl mezi "dokázané", "změřené", "strukturálně přesvědčivé" a "otevřené" budeme v knížce udržovat. Budujeme *věrný obrázek*, ne marketingové tvrzení.

1.5 Vizualizace: stejná hra, rozdílné výsledky

Diagram 1 ukazuje, proč hra "tři z deseti" dává legitimním stranám výhodu. Vlevo: Alice a Bob nezávisle vybírají po třech číslech a v 71 % případů má jejich průnik alespoň jeden prvek (na kterém se mohou shodnout). Vpravo: Eva také vybírá tři čísla, ale jen v 26 % případů obsahuje její trojice některé z čísel, která Alice s Bobem našli společně.

V této knížce uvidíme, jak se z této jednoduché ukázky stavebně buduje skutečný kryptografický protokol. Strukturální nevýhoda Evy v ilustrativním příkladu přechází ve skutečném



Obrázek 1: Hra "tři z deseti": všichni tři hráči nezávisle vybírají tři čísla z $\{1, \dots, 10\}$. Alice a Bob mají v 71 % případů alespoň jedno společné číslo (vlevo). Eva, ač má naprosto stejné zdroje (taky vybírá tři čísla), trefí některé z čísel, na kterých se Alice s Bobem shodli, jen v 26 % případů (vpravo). Poměr $71/26 \approx 2,7$ je strukturální výhoda legitimních stran.

protokolu na kombinaci argumentů — statistických (popsaných empiricky) a kryptografických (postavených na obtížnosti hashe SHA-256 vůči preimage). *Není* ekvivalentem matematicky dokázané "nezdolatelnosti"; spíše je to obrázek, kde má Eva strukturální nevýhodu a kde se dosud nikomu nepodařilo žádnou z testovaných útočnických strategií prosadit.

1.6 Co najdete v této knížce

Cesta v deseti kapitolách:

- **Kapitola 2** (Statická asymetrie): Zobecníme hru "tři z deseti" na velké prostory a m stran. Klíčový výsledek: *poměr asymetrie* $R \approx n/k$ (legitimní strany jsou R -krát zvýhodněny nad útočníkem). To je matematický základ, na kterém všechno stojí.
- **Kapitola 3** (Čtyři designová rozhodnutí): Statická asymetrie nestačí — potřebujeme protokol, který ji *zesílí přes mnoho kol*. Čtyři designová rozhodnutí (hashové adresování, drift, aktualizací pravidlo, AES bez paddingu) provedou tu transformaci. Tohle je srdce protokolu.
- **Kapitola 4** (Protokol v akci): Konkrétní implementace. Co se děje v jednom kole. 13 parametrů. Pět profilů (Reference, IoT, PQ128, Break-Demo, h_AB6).
- **Kapitola 5** (Statistický drift): Detail druhého designového rozhodnutí. Proč legitimní strany "vidí krajinu, ale jdou po nejřidších cestách".
- **Kapitola 6** (AES jako obrana proti zaplavení): Detail čtvrtého rozhodnutí. Proč protokol používá AES bez paddingu, a co to znamená pro útočníka s milionem kandidátních klíčů.
- **Kapitola 7** (Empirické důkazy): Co se reálně stalo, když jsme protokol pustili 50 tisíckrát. Jak prošel testy NIST. Jak se choval proti pěti třídám útočníků.
- **Kapitola 8** (Srovnání): Grata Cascade vs ECDH, ML-KEM, X-Wing hybrid, Merkleovy puzzly. Co je lepší, kde je horší, kde do hybridního stacku patří.
- **Kapitola 9** (Otevřené problémy): Co ještě nevíme. Sedm konkrétních otázek, na které ještě nemáme odpovědi.
- **Kapitola 10** (Glosář a kde dál): Slovník pojmů. Odkazy na hlavní paper, dokumentaci, kód.

Pokud chcete jen rychlý přehled (5 minut čtení), stačí kapitola **1**, **2** a poslední odstavec **6**. Pokud máte hodinu, přečtete **1-3** a **6**. Pokud chcete úplný obraz, projděte všechno.

2 Statická asymetrie: zobecněný problém narozenin

Co si odnést: hra "tři z deseti" se dá zobecnit. Pro n čísel a každého hráče vybírajícího k , mají Alice s Bobem nad útočnickem výhodu poměrem $R \approx n/k$ — a tento poměr je matematickou hranicí, kterou žádný výpočetní útok neposune. To je statický základ; samotný k protokolu nestačí.

V Sekci 1 jsme hráli hru "tři z deseti" a zjistili, že legitimní strany mají strukturální výhodu nad odposlouchávačem. Tato kapitola tu výhodu *změří* — pojmenuje ji a převede do tvaru, se kterým můžeme dál pracovat. Klíčový výsledek se vejde do tří symbolů: $R \approx n/k$.

2.1 Birthday paradox jako rozcvička

Pokud hodíte 23 lidí do jedné místnosti, pravděpodobnost, že alespoň dva z nich mají narozeniny ve stejný den, je přes 50%. To je pověsný "birthday paradox": při velikosti vesmíru $n = 365$ stačí jen $\sqrt{n} \approx 19$ vzorků, aby kolize byly pravděpodobné. Není to skutečný paradox — jen jsme intuitivně podceňovali, kolik je párů (každé dva lidi v místnosti generují jednu zkoušku, a $\binom{23}{2} = 253$ párů je zatraceně hodně).

Pro kryptografii je to zásadní princip. Vesmír velikosti n , k vzorků, a pravděpodobnost kolize roste přibližně jako k^2/n . Když je $n = 2^{128}$ (typický kryptografický prostor) a $k = 2^{40}$, máme $k^2/n = 2^{-48}$ — velmi nízká, ale ne nulová. Když $k = 2^{64}$, $k^2/n = 1$ — jistá kolize. Pro nás je důležité, že protokol *bezpečnostní hranice závisí na k^2/n* , nikoli na n samotném.

2.2 Co se stane se třemi stranami

Birthday paradox je o dvou množinách (dva výběry, jedna kolize). Co když máme tři? Alice, Bob a útočnice Eva, každá nezávisle vybírá k prvků z n . Eva chce trefit prvek, který si vybrali *oba* legitimní hráči. Pravděpodobnost, že Eva uspěje, není k^2/n — je k^3/n^2 , mnohem méně.

Proč? Protože její vlastní výběr přidá jeden další "filtr": prvek průniku $A \cap B$ se musí trefit i s jejím E . Každý další "rozumný hráč" v takovém problému přidá faktor k/n .

Pro matematicky zájemné: zobecněný birthday

Označme $P_m(n, k) = \Pr[A_1 \cap \dots \cap A_m \neq \emptyset]$, kde $A_1, \dots, A_m$ jsou nezávislé rovnoměrně náhodné k -podmnožiny $\{1, \dots, n\}$. Pro $k \ll n$ máme asymptotický odhad

$$P_m(n, k) \approx \frac{k^m}{n^{m-1}}.$$

Pro $m = 2$ (klasický birthday) to dává k^2/n . Pro $m = 3$ to dává $k^3/n^2 = (k/n) \cdot k^2/n =$ legitimní pravděpodobnost krát "filtr" k/n . Plný důkaz inkluze-exkluze a uzavřený tvar pro $m = 2$ jsou v paper v8, Věta `thm:pm-formula` a `thm:p2-closed`.

2.3 Poměr asymetrie: srdce této knížky

Tady přichází klíčový moment. Podívejme se na poměr toho, co dokáží legitimní strany ($P_2 \approx k^2/n$) a útočnice ($P_3 \approx k^3/n^2$):

$$R = \frac{P_2}{P_3} \approx \frac{k^2/n}{k^3/n^2} = \frac{n}{k}.$$

Tohle je *poměr asymetrie*, $R \approx n/k$. Je to číslo, které říká: *legitimní strany mají R -násobnou výhodu nad útočnickem na jedné události výběru*. A je to číslo velké — pokud je $n = 2^{128}$ a $k = 2^{40}$, dostáváme $R \approx 2^{88}$. Útočník je téměř 10^{27} -krát v nevýhodě.

$R \approx n/k$ je strukturální

Podstatné je, že R neroste s výpočetní silou útočníka. Není to "ECDH klíč o bitu, který útočník neuhodne". Je to vlastnost informace samotné: při daném k je vesmír velikosti n , a útočník ho prostě nemůže obsáhnout. Konstanta n/k je "kolik se útočník nemůže dozvědět".

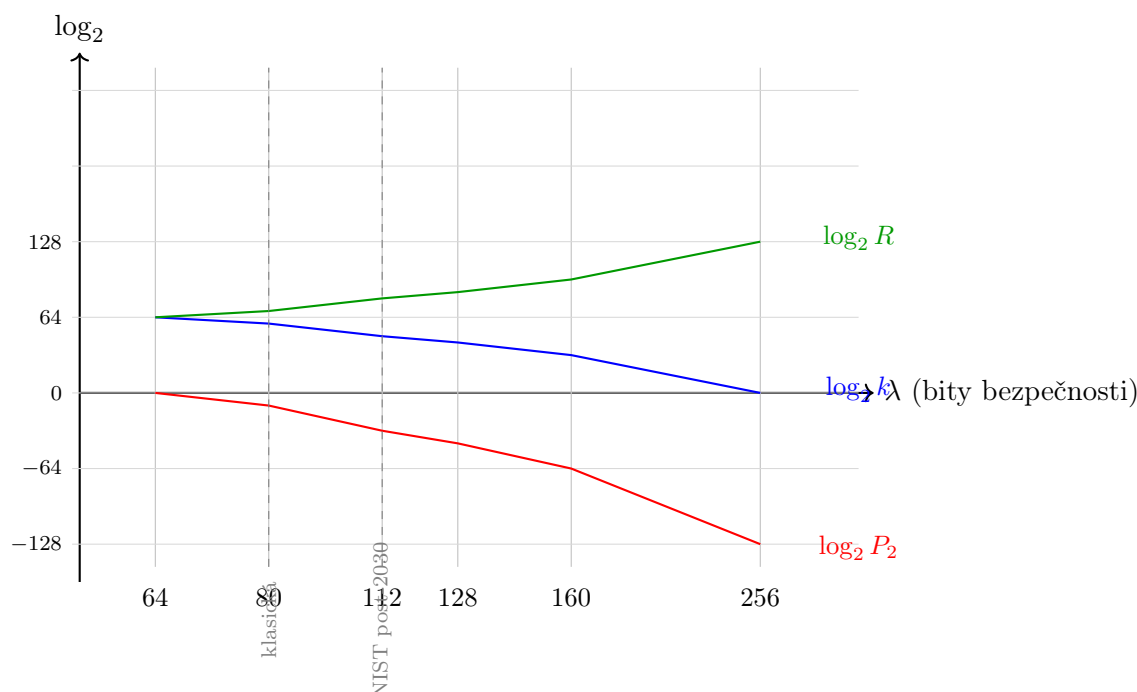
2.4 Volba parametrů: trojúhelník kompromisů

Co se stane, když si vybereme úroveň bezpečnosti? Řekneme: chceme, aby útočníkův úspěch P_3 byl menší než $2^{-\lambda}$ pro nějaké pevné λ (např. $\lambda = 80$ pro klasickou symetrickou bezpečnost). Pak z $P_3 = k^3/n^2 = 2^{-\lambda}$ vyplyne pevná hodnota k . A z té zase P_2 a R .

Tabulka 1 ukazuje, jak to vychází pro různé λ při $n = 2^{128}$. Všimněte si: zvýšením λ o 3 bity získáme 1 bit v R , ale zaplatíme 1 bitem v $\log_2 k$ a 2 bity v $\log_2 P_2$. To je podstata kompromisu — nelze "zlepšit všechno" naráz.

λ	Kontext	$\log_2 k$	$\log_2 P_2$	$\log_2 R$
64	slabý práh	64,00	0 (saturace)	64,00
80	klasická symetrická bezpečnost	58,67	-10,67	69,33
112	NIST post-2030	48,00	-32,00	80,00
128	plná 128bitová	42,67	-42,67	85,33
160	konzervativní	32,00	-64,00	96,00
256	extrémní (kosmologická)	0,00	-128,00	128,00

Tabulka 1: Volba parametrů pro $n = 2^{128}$ podle úrovně bezpečnosti λ . Tři veličiny tvoří lineární soustavu: každé zvýšení λ o 3 bity posune $\log_2 k$ o -1 , $\log_2 P_2$ o -2 a $\log_2 R$ o $+1$.



Obrázek 2: Trojúhelník kompromisů: jak se mění tři klíčové veličiny ($\log_2 k$, $\log_2 P_2$, $\log_2 R$) s rostoucí úrovní bezpečnosti λ . Modrá (k) klesá pomalu, červená (P_2) klesá nejrychleji, zelená (R) roste pomalu. Vidíme: vyšší bezpečnost $\Rightarrow$ menší pool, menší legitimní kolize, větší asymetrie. Žádné dvě nelze "zlepšit" současně.

2.5 Proč je toto všechno *statické*

Důležité je si uvědomit, co výše uvedené říká *a co ne*. Říká: pokud Alice, Bob a Eva každý nezávisle a jednorázově vyberou své k -prvkové množiny, bude poměr šancí dán $R \approx n/k$. To je *jediná událost*.

Co ale neříká: jak to funguje, když protokol běží *přes mnoho kol*, kdy se interní stavy stran vyvíjejí, kdy si stavy vyměňují signály, kdy útočník akumuluje pozorování. To je *dynamický režim*, a statická analýza pro něj nestačí.

Pro praktický kryptografický protokol je to problém: pokud bychom zvolili $\lambda = 80$ a tedy $k = 2^{58,67}$, $P_2 \approx 2^{-10,67}$, znamená to, že na jednu událost výběru by Alice s Bobem měli kolizi pravděpodobností přibližně $1/1\,627$. To je hrozné — protokol by selhal v 99,94% případů. Pokud bychom zvolili k tak velké, aby kolize byla jistá ($k^2 \approx n$, tj. $k \approx 2^{64}$), útočnickova šance vyletí na $k^3/n^2 = 2^{-128} \cdot k = 2^{-64}$ — nevýhodný kompromis.

Statika sama nestačí

Statická hranice nám dává *strop* útočnickovy znalosti, ne praktický protokol. Praktický protokol musí najít způsob, jak *spolehlivě* (téměř 100%) najít prvek průniku, aniž by zhoršil R . Jinými slovy: musí transformovat statickou jednorázovou událost na dynamický opakovaný proces, který asymetrii *zachová a rozšíří*.

A přesně to dělají čtyři designová rozhodnutí, kterým je věnována Kapitola 3.

Statická asymetrie říká, jaká je strukturální výhoda. Dynamický systém ukáže, jak ji uchopit.

3 Čtyři designová rozhodnutí

Co si odnést: protokol stojí na čtyřech designových rozhodnutích, která dohromady transformují statickou asymetrii na dynamický systém. (1) Hash jako jednosměrný commitment, (2) veřejný drift přes náhodný vektor, (3) aktualizací pravidlo bez gradientního úniku, (4) AES bez paddingu pro flooding obranu. Každé z nich blokuje jiný útočnický vektor; spolu vytvářejí kaskádu.

V Sekci 2 jsme narazili na praktický problém: statická asymetrie $R \approx n/k$ existuje, ale pro praktický protokol je bezcenná — s reasonable λ by se Alice s Bobem v 99,94% případů ani neshodli na jediném prvku průniku. Potřebujeme tedy něco, co statickou asymetrii *zachová* (útočník zůstane v nevýhodě) a zároveň *zesílí* (legitimní strany dosáhnou téměř jisté shody).

To je transformační problém Grata Cascade. Dělí se na tři dílčí podproblémy:

1. **Komunikace bez prozrazení.** Strany si musí veřejně vyměnit dost informací, aby zjistily, co mají společného, ale ne tolik, aby to mohla zjistit i Eva.
2. **Útočník zůstává na statické hranici.** Každé další kolo komunikace nesmí Evu dostat do lepší pozice než tu danou poměrem R . Žádné akumulační efekty, žádné úniky v čase.
3. **Klíč vyžaduje znalost průniku.** Finální kryptografický klíč musí být odvoditelný *pouze* ze samotných prvků průniku — ne z toho, co vidíme na kanále.

Grata Cascade tyto tři podproblémy řeší *čtyřmi* designovými rozhodnutími. Každé z nich je samo o sobě jednoduché; vykouzlit z nich *spolu* funkční CKA protokol je netriviální. Pojdme po jednom.

3.1 Rozhodnutí 1: Hash jako slibovací zámek

Problém. Alice má v paměti svůj privátní pool vektorů $\mathcal{V}_A$ (řekněme tisíce 32bytových vektorů). Bob má svůj $\mathcal{V}_B$. Chtějí si vyměnit informaci o tom, co mají *společné*, aniž by ten společný materiál unikl do veřejného kanálu. Jak?

Řešení. Alice spočte SHA-256 hashe všech svých vektorů a po kanále pošle *prefixy* těchto hashů (např. prvních 5 bytů — ne celý hash). Bob zhashuje vlastní vektory a hledá, jestli některý z nich má prefix shodný s tím, co dostal. Pokud ano, je to kandidát na společný vektor.

Hash je jednosměrný commitment

SHA-256 je *jednosměrný*: ze vstupu spočtu hash snadno, ale obráceně to nejde. Když Alice pošle $h(v) = \text{"f3a2..."}$ prozradí to o v téměř nic, kromě faktu, že existuje. Ale když Bob má svůj vlastní v' a spočte $h(v') = \text{"f3a2..."}$, okamžitě ví, že $v = v'$. Veřejně se nic neprozradilo, soukromě se shodli.

Nuance: zkrácený hash. Alice neposílá celých 32 bytů hashe, ale jen prvních pár. Proč?

- **Plný hash** (32 bytů): Bob skoro nikdy nenajde shodu náhodou, ale Alice musí přenášet $32 \cdot |\mathcal{V}_A|$ bytů na kolo — nákladné na objem přenášovaných dat.
- **Krátký hash** (např. $h_{AB} = 5$ bytů): Bob najde víc *kandidátních* shod (některé jsou náhodné, falešně pozitivní), ale objem přenosu je $5/32$. Z náhodných shod se *kaskádově* odfiltrují v dalších rozhodnutích (3 a 4).

V referenční konfiguraci v2 používáme tři různé délky pro tři role:

- $h_{AB} = 5$ bytů: prefix, který Alice posílá Bobovi (krok $A \rightarrow B$).
- $h_{BA} = 2$ byty: prefix, který Bob posílá zpět Alici (krok $B \rightarrow A$) jako "našel jsem tyto kandidáty".
- $h_P = 8$ bytů: prefix pro *password* vektory (cross-round vektory, dlouhodobější význam).

Součtem dostáváme **kumulativní preimage bariéru** $\lambda = 8 \cdot (5 + 2 + 8) = 120$ bitů. Tolik bitů musí útočník "uhodnout" navíc nad jeden vektor, aby celou kaskádu prošel. To je obrovská hradba.

Historický kontext: Merkleho puzzle

Princip "domluva klíče přes veřejnou hashovací funkci" pochází z Merkleho puzzle (1978): Alice pošle N hádanek po $O(N)$ práci, Bob jednu vyřeší v $O(N)$, Eva musí prozkoumat všechny v $O(N^2)$. Impagliazzo a Rudich (1989) ukázali, že $O(N^2)$ je v black-box modelu optimální. Grata Cascade tuto linii dále rozvíjí: kombinací s ostatními rozhodnutími se konjekturálně posouvá útočnickova práce nad $O(N^2)$. Plnou diskusi viz paper v8 §4.2.

3.2 Rozhodnutí 2: Veřejný drift přes náhodný vektor

Problém. Alice a Bob mají své privátní pooly. Po prvním kole už něco vědí o sobě navzájem. V dalším kole *musí* svoje pooly nějak aktualizovat — jinak by se opakovali a celá hra by byla tatáž. Ale jak je aktualizovat, aby:

- Pooly Alice a Boba zůstaly *korelované* (jinak nebudou mít na čem se shodnout)?
- Eva *neviděla*, které vektory každý z nich má?

Řešení. V každém kole se vygeneruje *veřejný náhodný vektor* R_t (běžný nějakou kryptograficky náhodnou rutinou). Alice ho vidí, Bob ho vidí, Eva ho také vidí. Alice i Bob aplikují identické pravidlo aktualizace, které R_t *posílí* jejich pooly. Eva, ač R_t vidí, *neví, které konkrétní vektory* Alice nebo Bob měli, takže nemůže pooly zrekonstruovat.

Všichni vidíme stejnou mapu, každý jdeme jinou cestou

R_t je veřejná. Pravidlo aktualizace je veřejné. Stat distribuce, která z toho vznikne, je veřejná — Eva ji může spočítat přesně. Co *není* veřejné: konkrétní cesty, kterými Alice a Bob touto distribucí prošli. Mají identickou mapu, ale vidí ji "ze své pozice".

To je jemný, ale klíčový bod. Eva si dokáže udělat dokonalý obraz *statistického prostoru*, ale ne *konkrétních bodů* v něm, které stranám patří. To znamená, že může útočit jen pravděpodobnostně — a pravděpodobnost trefit konkrétní vektor zůstává $1/n$, protože vektor sám se vyvíjí podle R_t , který je rovnoměrně náhodný.

Pravděpodobnostní aktualizace. Druhá vrstva ochrany: aktualizace každého vektoru proběhne s pravděpodobností $p_{\text{upd}} < 1$. V referenci v2 je $p_{\text{upd}} = 0,75$. To znamená, že některé vektory se v daném kole *neaktualizují*. Eva sice ví, že 75% vektorů bylo aktualizováno, ale *neví které*. Statistická maska, která útočníkovi schovává konkrétní stav.

3.3 Rozhodnutí 3: Aktualizační pravidlo (inspirace TPM)

Problém. Když máme veřejný R_t a chceme každou stranu nechat aktualizovat své vektory, jaké konkrétní pravidlo použít? Musí splnit dvě podmínky: (a) musí pooly Alice a Boba postupně přibližovat (jinak se nikdy neshodnou), (b) nesmí prozradit gradient (kterým směrem se který vektor pohnul).

Inspirace. Grata Cascade čerpá z historického protokolu **Tree Parity Machine** (TPM, 2002), ve kterém dvě neuronové sítě postupně synchronizovaly své váhy. TPM byl ale později prolomen *geometrickým útokem* (Klimov-Mityagin-Shamir 2003): útočník mohl ze synchronizačních zpráv rekonstruovat váhy díky úniku gradientu.

Grata Cascade si bere strukturu (iterativní synchronizace přes sdílený signál), ale *neúnikne gradient*. Pravidlo je deterministické a jednoduché:

$$v_i \leftarrow \begin{cases} v_i + R_i - 128 & \text{pokud výsledek je v rozsahu } [0, 255], \\ R_i & \text{jinak (overflow).} \end{cases}$$

(Každá komponenta vektoru je byte $\in [0, 255]$. Posun o $R_i - 128$ centruje aktualizaci kolem nuly: někdy posune nahoru, jindy dolů.)

Tančíme na stejnou hudbu, jiná choreografie

Alice a Bob slyší stejnou píseň (R_t). Aplikují stejné taneční pravidlo. Ale jejich startovací pozice (privátní vektory) jsou různé, takže končí v různých místech. Eva slyší píseň, ale neví, kde Alice a Bob začali — takže nemůže říct, kde končí.

To je strukturální rozdíl od TPM: tam synchronizace mezi sítěmi prozradila gradient (kterým směrem se váhy pohly). Tady se s žádnou sdílenou informací mezi Alice a Bobem nepracuje. Jen oba aplikují identickou veřejnou aktualizaci.

3.4 Rozhodnutí 4: AES bez paddingu (žádné validační orákulum)

Problém. Po několika kolech mají Alice a Bob seznam kandidátních "password vektorů" — vektory, které prošly všemi filtry a jsou pravděpodobně součástí jejich průniku. Z těch vektorů musí oba odvodit stejný *finální klíč* K^* .

Naivní řešení: Alice spočte hash $K^* = \text{SHA-256}(v_1 \| v_2 \| \dots \| v_M)$ a Bob udělá totéž. Pokud mají stejné vektory, dostanou stejný klíč. Funguje to.

Co když ale Eva má pool 10^{15} kandidátních vektorů? Mohla by zkusit miliony kombinací, spočítat hashe, a možná najít K^* . Jak ji odradit?

Řešení. Místo toho, aby Alice posílala vektory přímo, šifruje je AES v režimu CBC *bez paddingu* (`PaddingMode.None`). To má dvě klíčové vlastnosti:

1. **Žádné validační orákulum.** U AES s normálním paddingem (PKCS7) lze špatný klíč detekovat: dešifrování produkuje neplatný padding, což hru přerve. *Bez paddingu* dešifrování špatným klíčem produkuje pseudonáhodné bytes, nerozlišitelné od korektního výsledku. Eva nemá způsob, jak ověřit, jestli její odhad klíče je správný.
2. **Nerozlišitelná kandidátní množina.** Pro 32-byte vektor je pravděpodobnost, že náhodný kandidát "vypadá rozumně", $2^{-\lambda} = 2^{-120}$. Ale celkový prostor možných vektorů je $2^{8 \cdot 32} = 2^{256}$. Po preimage bariéře zbývá $2^{256-120} = 2^{136}$ kandidátů, kteří všichni *vypadají stejně dobře*. Eva nemůže žádného z nich zvýhodnit.

Hromada identických klíčů, mnoho slepých chodeb

Představ si svazek 2^{136} identických klíčů. Všechny otevírají *stejně dveře*, ale za dveřmi se cesty rozdělují — každý klíč vede do své vlastní chodby (celkem 2^{136} chodeb). Slepé jsou všechny až na jednu jedinou, která vede k tajemství. Eva svým klíčem dveře otevře a ocitne se v nějaké chodbě — holé zdi, žádné značky, žádný způsob, jak poznat, jestli je slepá, nebo není. Všechny chodby vypadají identicky. AES bez paddingu vyrábí přesně tuhle situaci: každý odhad klíče něco dešifruje, ale výstupem je 32 bajtů pseudonáhodného šumu, neodlišitelného od šumu z jiných odhadů.

3.5 Kombinovaný systém: jak to spolu funguje

Tato čtyři rozhodnutí spolu *kaskádově* zajistí transformaci ze statického na dynamický režim. Každé rozhodnutí adresuje jeden specifický vektor útoku:

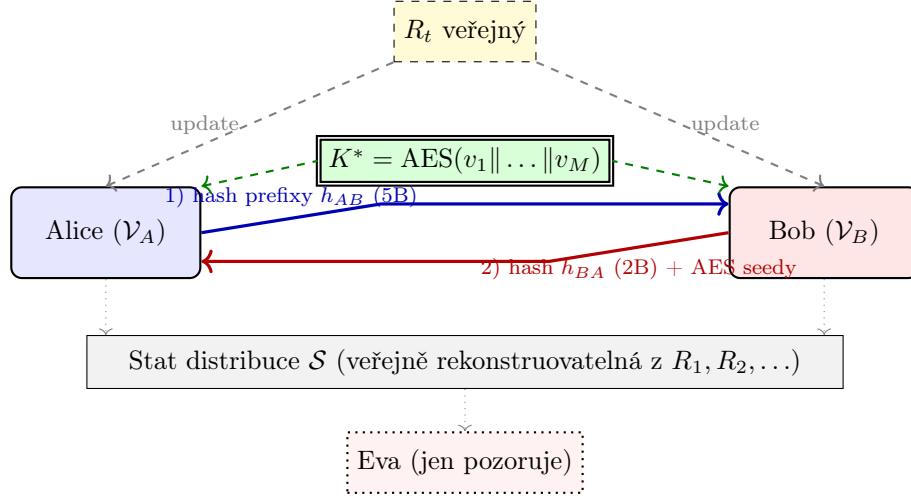
- **Hash addressing** chrání před přímým výpočetním útokem: i s neomezeným časem nelze obrátit SHA-256.
- **Veřejný drift** chrání před akumulacími útoky: Eva sice vidí všechny R_t , ale konkrétní pooly stran zůstávají neviditelné.
- **Aktualizační pravidlo** chrání před gradientním únikem (TPM lekce): synchronizace nepředá Evě informaci o trajektoriích.
- **AES bez paddingu** chrání před zaplavením kandidáty: ani s milionem vektorů nemá Eva validační orákulum.

V Kapitole 4 se na konkrétní implementaci podíváme detailněji: co dělá Alice v kroku 1, co posílá Bob v kroku 2, jak vypadá Stat tabulka, a jak se z toho odvozuje finální klíč. Kapitoly 5 a 6 pak vyjmou rozhodnutí 2+3 (drift) a 4 (AES) na detailní mikroskop.

Statika dává hranici. Drift dává dynamiku. Hash dává sílu jednosměrnosti. AES bez paddingu dává nerozlišitelnost. Dohromady: konvergence a bezpečnost.

Aspekt	Statický (Birthday)	Dynamický (Grata Cascade)
Výběr	Náhodný, jednorázový	Iterativní, řízený driftem
Vesmír	Pevný, $ U = n$	Efektivně zúžený přes Stat
Korelace stran	Žádná	Synchronizovaná přes R_t
Cíl	Průnik množin	Specifická posloupnost hash hitů
Pozice útočníka	Třetí strana ve hře	Pozorovatel bez koherentního stavu

Tabulka 2: Statický model birthday paradox versus dynamický systém Grata Cascade. Dynamické mechanismy statický základ *posilují*, ale neporušují — útočník zůstává na poměru asymetrie $R \approx n/k$ jako dolní mezi výhody legitimních stran.



Obrázek 3: Kombinovaný dynamický systém v jednom kole. Veřejný R_t aktualizuje pooly obou stran identicky. Alice odešle Bobovi hash prefixy h_{AB} (rozhodnutí 1, krok 1). Bob vrátí hashe h_{BA} a AES-šifrované seedy (rozhodnutí 1 + 4, krok 2). Stat distribuce je rekonstruovatelná Evou, ale konkrétní pooly nikoli (rozhodnutí 2). Po několika kolech Alice i Bob zfinalizují K^* z dostupných password vektorů (rozhodnutí 4).

4 Protokol v akci

Co si odnést: jedno kolo má pět kroků (A pošle hashe, B najde kandidáty a vrátí klíč, A dešifruje, oba aktualizují Stat, případně terminace). Konvergence trvá obvykle ~ 49 kol. Z 13 parametrů stačí znát šest strukturálních ($L, N, h_{AB}, h_{BA}, h_P, M$); zbylých sedm jsou kalibrační detaily.

V Kapitole 3 jsme rozebrali *co* čtyři designová rozhodnutí dělají. Tato kapitola ukazuje *jak* se uplatňují v jednom kole protokolu: kdo komu co posílá, v jakém pořadí, s jakými parametry. Cíl: po přečtení by měl být čtenář schopen vyjít vstříc kódu nebo paperu v8 §5 a vědět, čeho se každý parametr týká.

4.1 Parametry: co je třeba znát, co je vnitřní detail

Reference v2 má 13 parametrů. Pro porozumění protokolu jich ale stačí znát šest — ty popisují *strukturu* (velikost vektoru, velikost poolu, délky hashů, počet password slotů). Zbývajících sedm je vnitřní/kalibrační detail (prahy, pravděpodobnost aktualizace, stropy iterací). Rozdělil jsem je do dvou tabulek.

Strukturální parametry.

Vnitřní a kalibrační parametry.

Symbol	Hodnota	Co určuje
L	32 B	Délka jednoho vektoru. Větší $\Rightarrow$ exponenciálně více nerozlišitelnosti, větší paměť.
N	4096	Vektorů v poolu na stranu. Větší $\Rightarrow$ více kandidátů na shodu, větší objem dat.
h_{AB}	5 B	Délka hashe v kanálu A $\rightarrow$ B. Krátký $\Rightarrow$ tichá selhání. Dlouhý $\Rightarrow$ objem dat.
h_{BA}	2 B	Délka hashe v kanálu B $\rightarrow$ A. <i>Záměrně krátký</i> (viz §6).
h_P	8 B	Délka password hashe (Final fáze). Identifikátor přes mnoho kol.
M	8	Počet password kandidátů na kolo. Větší $\Rightarrow$ útočnickova kombinatorika roste.

Tabulka 3: Šest strukturálních parametrů. Stačí znát na pochopení protokolu. Tučné hodnoty jsou empiricky kalibrované. Součet $h_{AB} + h_{BA} + h_P = 15$ B $\Rightarrow$ **kumulativní preimage bariéra** $\lambda = 120$ bitů (viz Kap. 3).

Symbol	Hodnota	Role
s	4	Rozsah perturbace seedu ($\pm s$). Empiricky neutrální v [1, 30].
p_{upd}	0,75	Pravděpodobnost aktualizace vektoru za kolo. Vynucený rozsah [0,6, 0,95].
τ	-512	Práh terminace ($\log_2$). Pod ním se sezení ukončí.
θ_{cut}	-8	Práh poolu. V ref v2 strukturálně neaktivní.
θ_{cand}	-8 (vypnut)	Filtr kandidátů (volitelná Tier-3 obrana).
MaxIter	500	Strop iterací. Medián konvergence ref v2 ≈ 49 .

Tabulka 4: Sedm vnitřních a kalibračních parametrů. Pro pochopení protokolu nejsou nutné, ale jsou důležité pro běh. Plná detail viz `configs/reference.json` a paper v8 main §5.

4.2 Pět parametrických profilů

Jeden protokol, různé situace — proto Grata Cascade tvoří *rodinu* konfigurací, ne jednu fixní implementaci. V repu jsou aktuálně tyto:

- **reference** (v2): produkční baseline, plně empiricky validován.
- **low_resource_iot**: pro IoT zařízení, $L = 16$, $N = 1024$ — méně paměti, slabší preimage ($\lambda = 96$ bitů).
- **high_security_pq128**: paranoidní postkvantová varianta, $L = 64$, $N = 8192$, $h_{AB} = 6$ — 128-bitová bezpečnost, ale ~ 64 s na kolo (vs. ~ 1 s pro reference).
- **break_demo** (výzkumný): *záměrně oslabená* verze pro pedagogickou demonstraci útoků, $L = 8$, $h_{AB} = 3$, $\lambda = 56$ bitů. Ne pro produkci.
- **reference_h_AB6** (v3 kandidát): klon reference s $h_{AB} = 5 \rightarrow 6$ ($\lambda = 128$ bitů). Validovaný 50k-run ablací (viz §7).

Profil **reference_h_AB6** je nejnovější a nejnadějnější: posílí λ na 128 bitů za cenu +4 KB objemu dat na kolo, validačně dosahuje 0 divergencí na 50k běhů. Pokud uživatel přijme tento kompromis, je to vhodný kandidát na "v3 reference".

4.3 Co se přesně děje v jednom kole

Pojďme po jednotlivých krocích jednoho kola t . Předpokládáme, že máme inicializované pooly $\mathcal{V}_A$, $\mathcal{V}_B$ (každý 4096 náhodných 32bytových vektorů) a inicializovanou Stat tabulku $\mathcal{S}$ (rovnoměrná).

Krok 1: A posílá hashe. Alice spočte SHA-256 hash každého vektoru ze svého poolu a odřízne prvních 5 bytů (h_{AB}). Výsledný seznam $\{H[0:5](v) : v \in \mathcal{V}_A\}$ obsahuje 4096 hashových prefixů, každý 5 B. To je $4096 \cdot 5 = 20$ KB — největší zpráva v celém kole. Pošle Bobovi.

Krok 2: B hledá kolize. Bob projde svůj pool, hashuje vlastní vektory a hledá prefixy, které odpovídají těm přijatým od Alice. To dá nějaký počet *kandidátních kolizí*. Z nich:

1. Seřadí vzestupně podle pravděpodobnosti pod Stat $\mathcal{S}_B$ (nejmenší pravděpodobnost první — to je tail selekce, viz §5).
2. Volitelně filtruje prahem θ_{cand} (v reference vypnuto).
3. Vybere prvních $M = 8$ kandidátů.

Tito kandidáti jsou *pravděpodobně* v průniku s Aliciným poolem (jejich hashe se shodují) a *pravděpodobně* jsou statisticky neobvyklí (tail).

Krok 3: B posílá zpět. Bob posílá Alici tři věci:

- Veřejný náhodný vektor R_t (32 B) — vstup do Stat aktualizace.
- Hashe B→A prefixů těchto kandidátů: $\{H[5:7](v) : v \in \text{kandidáti}\}$ ($M \cdot h_{BA} = 16$ B).
- AES-šifrovaný seed σ_t , pod klíčem $K_t = H(v_{\pi(1)} \parallel \dots \parallel v_{\pi(M)})$ (32 B).

Krok 4: A dešifruje a aktualizuje. Alice enumeruje, které kombinace 8 vektorů ze svého poolu odpovídají přijatým B→A hashům. Pro každou kandidátní kombinaci spočte K'_t , pokusí se dešifrovat AES (*vždy "uspěje" syntakticky*, kvůli NoPadding) a získá kandidátní seed σ'_t . Bez paddingu nemá způsob, jak ověřit, který je správný — pokračuje s několika kandidáty.

Pak: obě strany aplikují *deterministickou* aktualizaci $\mathcal{S}$ pomocí R_t a aktualizují své pooly s pravděpodobnostmi $p_{\text{upd}} = 0,75$ (perturbace $\pm s = \pm 4$, filtrované přes θ_{cut}).

Krok 5: Terminace. B udržuje akumulovaný seznam $\mathcal{P}$ password vektorů (těch, které se v kandidátní množině objevily). Když součet $\log_2 \Pr_{\mathcal{S}_B}(v)$ přes $v \in \mathcal{P}$ klesne pod $\tau = -512$, končíme. B pošle Final zprávu obsahující $h_P = 8$ -bytové hashe všech $|\mathcal{P}|$ password vektorů.

A enumeruje kombinace svých kandidátních seedů, hledá ty, jejichž h_P hash odpovídá. Ko-nečný klíč:

$$K^* = H(v_{\pi(1)} \parallel v_{\pi(2)} \parallel \dots \parallel v_{\pi(|\mathcal{P}|)}).$$

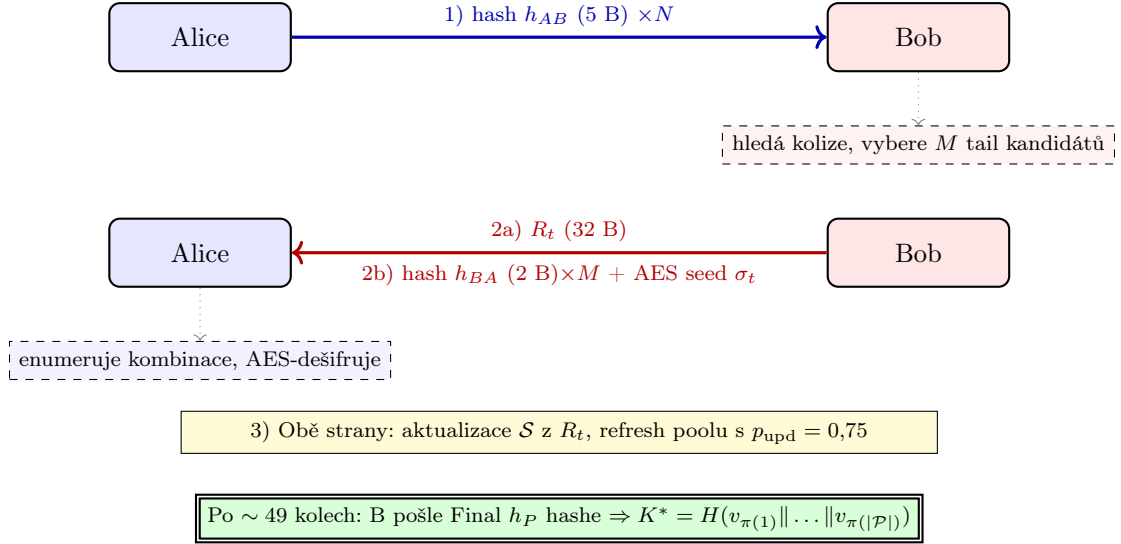
Mediánové sezení má ≈ 49 kol. Objem dat na kolo je $\approx 20,5$ KB. Celé sezení: zhruba 1 MB.

4.4 Vizualizace: jedno kolo přehledně

4.5 Tři role hashů, tři různá kalibrační omezení

Tři hashové délky h_{AB} , h_{BA} , h_P *nejsou* jen kalibrační knoflíky se sdílenou "bezpečnostní" sémantikou. Každá slouží jiné roli a kalibruje se proti jinému omezení:

- **h_{AB} jako práh správnosti.** Příliš krátký h_{AB} způsobí, že náhodné B-vektory se náhodou shodují s A-hashi (falešné kolize) a Bob staví seed na špatných vektorech. To je *tiché selhání* bez možnosti zotavení. Kalibrační vzorec: $h_{AB} \geq \lceil (2 \log_2 N + 8) / 8 \rceil$. Pro $N = 4096$ to dává $h_{AB} \geq 4$, ref používá 5 (s 8-bit marží).
- **h_{BA} jako záměrně permisivní filtr.** *Kratší je lepší.* Delší h_{BA} by způsobil, že útočník s velkým poolem nakonec dostane jediný jednoznačný kandidát, který může verifikovat. Krátký h_{BA} *záměrně* udržuje, aby přes první dva hashové filtry (h_{AB} a h_{BA}) prošlo i pro extrémní pooly více než jeden kandidát — útočník dostává *záplavu* kandidátů, ne specifickou identifikaci. Detail v §6.



Obrázek 4: Anatomie jednoho kola. Krok 1: Alice pošle 4096 zkrácených hashů (20 KB). Krok 2: Bob hledá kolize, vybere $M = 8$ tail kandidátů, pošle zpět R_t + hashe h_{BA} + AES seed. Krok 3: oba aktualizují Stat. Po ~ 49 kolech terminace.

- **h_P jako synchronizace přes kola.** Po ~ 50 kolech má Alice akumulovaný kandidátní pool ~ 400 vektorů. h_P musí být dostatečně dlouhý, aby Final hashe jednoznačně identifikovaly password vektory bez tichých neshod klíče. Pro $h_P = 8$ B a 50-kolové sezení: očekávaná tichá neshoda $\approx 1,7 \cdot 10^{-16}$ na sezení.

Třemi různými omezeními se vysvětluje, proč ref v2 nemá $h_{AB} = h_{BA} = h_P = 8$ B (jak by si naivně přál uživatel) — to by bylo $\lambda = 192$ bitů, ale $h_{BA} = 8$ B by oslabilo flooding obranu na "specifická identifikace" a zničilo design. Per-role kalibrace je strukturální, ne kosmetická.

4.6 Model útočníka

Pro úplnost: koho předpokládáme jako útočníka.

Grata Cascade je navržena pro *pasivní útočníky*: ti vidí všechny veřejné zprávy (transkripci, R_t vektory, AES šifry, Final hashe), mají PPT (probabilistic polynomial time) výpočetní zdroje. Útočníkem může být i kvantový počítač.

Mimo scope: aktivní síťový útočník, který by mohl modifikovat zprávy v transitu. Grata Cascade se má provozovat *uvnitř* autentizovaného transportu (TLS, Noise framework). Tam transportní vrstva zajistí integritu zpráv a autentizaci protistrany. Aktivní útočník, který obejde transport, má striktně větší přístup než cokoli, co by konkrétní útok na Grata Cascade přidal.

V Kapitole 7 uvidíme *pět konkrétních tříd* pasivních útočníků (B.1, B.2, B.3, B.4, B.7), proti kterým protokol obstál.

5 Statistický drift

Co si odnést: Stat distribuce je veřejná (i útočník ji umí spočítat), ale konkrétní Bobovy výběry z ní nikoli. Bob vybírá z „tailu“ — vzácných hodnot, kde útočníkův přímý sampling téměř netrefí. Pravděpodobnost aktualizace $p_{\text{upd}} = 0,75$ má úzký funkční rozsah $[0,6, 0,95]$; mimo něj protokol buď nekonverguje, nebo se Stat zhroutí.

V Kapitole 3 jsme řekli, že "veřejný drift přes náhodný vektor" je rozhodnutí 2. Tady se na něj podíváme detailněji: jak vypadá ta sdílená statistická distribuce $\mathcal{S}$, proč legitimní strany volí "tail" kandidáty, a proč existuje úzký operační rozsah p_{upd} .

5.1 Stat: veřejně rekonstruovatelná mapa

Každá strana (Alice i Bob) si lokálně udržuje tabulku $\mathcal{S}$. Pro každou z $L = 32$ pozic vektoru je v ní jedna pravděpodobnostní distribuce nad 256 možnými hodnotami bytu. Konkrétně: $p_i(x)$ říká, jak je v pozici i pravděpodobné, že byte má hodnotu x . Na začátku jsou všechny distribuce rovnoměrné — $p_i(x) = 1/256$ pro všechna x (žádná hodnota není výjimečná).

Postupem kol se distribuce *koncentrují*: některé hodnoty bytu získávají vyšší pravděpodobnost, jiné klesají. To je ten "drift" v názvu kapitoly. Pojdme se podívat, jak to funguje.

Pravidlo aktualizace. Když přijde nový R_t , obě strany pro každou pozici i provedou tři kroky:

1. Vezmou starou distribuci p_i (256 čísel sčítajících se k 1).
2. Vyrobit *posunutou* distribuci $\tilde{p}_i$. Posun přímo zrcadlí pravidlo aktualizace vektorů z §3.3: pokud měl vektor v pozici i hodnotu v , po aktualizaci má $v + R_{t,i} - 128$ (pokud výsledek leží v $[0, 255]$) nebo $R_{t,i}$ (přetečení). $\tilde{p}_i(x)$ je pravděpodobnost, že po této operaci skončí byte na hodnotě x .
3. Smíchají v poměru $p_{\text{upd}} : (1 - p_{\text{upd}})$:

$$p'_i(x) = p_{\text{upd}} \cdot p_i(x) + (1 - p_{\text{upd}}) \cdot \tilde{p}_i(x).$$

Pro reference v2 je $p_{\text{upd}} = 0,75$, takže *starou* distribuci máme s váhou 75 % a *posunutou* s váhou 25 %.

Pravděpodobnost celého vektoru v je pak součin přes pozice: $\Pr_{\mathcal{S}}(v) = \prod_i p_i(v_i)$.

Konkrétní příklad posunu

Řekněme, že na pozici i je $R_{t,i} = 200$. Posun je $200 - 128 = +72$. Pak:

- Hmota, která byla na hodnotách 0–183, se přesune na hodnoty 72–255 (posun o +72).
- Hmota na hodnotách 184–255 by tekla mimo bytový rozsah (256–327). Místo toho ji přetečení posbírá a celou ji nasype na hodnotu 200 ($= R_{t,i}$).
- Výsledná posunutá distribuce $\tilde{p}_i$ má tedy první kus posunutý a druhý "spadnutý" do jednoho bodu. Pak ji smícháme v poměru 0,75 : 0,25 se starou.

Drift přes mnoho kol je akumulace těchto posunů a přetečení. Začínáme rovnoměrně, ale přetečení s každým R_t vlije malou kapku hmoty na specifickou hodnotu, a po desítkách kol se tyto kapky skládají do struktury, kterou vidíme v Diagramu 5.

Veřejnost Stat $\neq$ kompromitace

$\mathcal{S}$ je *deterministickou funkcí* veřejné posloupnosti $R_1, R_2, \dots$. Útočník ji může spočítat *přesně* stejně jako Alice s Bobem. Tohle není problém. Problém by byl, kdyby útočník věděl *které vektory* Alice a Bob aktualizovali. Ale to neví, protože aktualizace probíhá s pravděpodobností $p_{\text{upd}} < 1$ na každý vektor nezávisle.

Tahle distinkce mezi "distribuční znalostí" (sdílená všemi včetně útočníka) a "trajektoriovou znalostí" (privátní pro každou stranu) je klíčová. Útočník vidí rozsahovou mapu, ale ne, kde Alice a Bob konkrétně stojí.

5.2 Inverzně-pravděpodobnostní výběr

Když Bob v kroku 2 (viz §4.3) vybírá $M = 8$ password kandidátů z hashových kolizí, řadí je vzestupně podle pravděpodobnosti pod $\mathcal{S}_B$ a vybírá *nejnižší*. To znamená kandidáty s *nejmenší* pravděpodobností v aktuální distribuci, tedy ty z "tailu".

Proč tail a ne typický střed? Představte si *kolo štěstí* se 256 sektory — po jednom pro každou hodnotu bytu. Sektory ale nemají stejnou velikost: jejich plocha je úměrná pravděpodobnosti $p_i(x)$ z distribuce $\mathcal{S}$. Některé sektory jsou tlusté klíny (typické hodnoty s vysokou p), jiné tenké škvírky sotva viditelné (tail hodnoty s nízkou p). Tohle kolo si umí postavit kdokoli, kdo zná veřejnou posloupnost R_t — tedy i útočník.

- **Útočník** kolem zatočí: kulička skončí v sektoru úměrně jeho velikosti, takže drtivou většinu času spadne do nějakého tlustého klínu. Tomuhle se říká *přímý sampling* z $\mathcal{S}$.
- **Bob** kuličku *nehází*, on *cíleně ukáže prstem na nejtenčí škvírku*. Konkrétní vektor v , který si vybere, má pod $\mathcal{S}$ pravděpodobnost $\approx 2^{-160}$ — naprosto nepatrná škvíra v jinak tlustých klínech.

To je inverzně-pravděpodobnostní obranný princip: Bobova a útočnickova strategie jsou *záměrně protisměrné*. Ze stejného kola štěstí každý cílí na opačný konec.

Stejné kolo, opačné strategie

Distribuce $\mathcal{S}$ je veřejně rekonstruovatelné kolo, které vidí všichni stejně. Bob ale ukazuje na nejtenčí škvírku, zatímco útočníkův nejlepší přímý sampling skončí v tlustém klínu. Aby útočník trefil Bobovu volbu bez šťastné náhody, musel by místo zatočení *enumerovat* škvírky — a těch je ve 32-bytovém prostoru exponenciálně mnoho.

Pro neaktivní filtr θ_{cand} (default reference v2) jsou typická čísla taková: medián $\log_2 \Pr$ Bobových kandidátů je ≈ -160 bitů, zatímco útočníkův přímý sampling z $\mathcal{S}$ trefí v průměru hodnoty s $\log_2 \Pr \approx -75$. Mezi -75 a -160 je propast 2^{85} — tolik je řádově "vzdálenost" mezi typickou a tail oblastí v 32-bytovém prostoru.

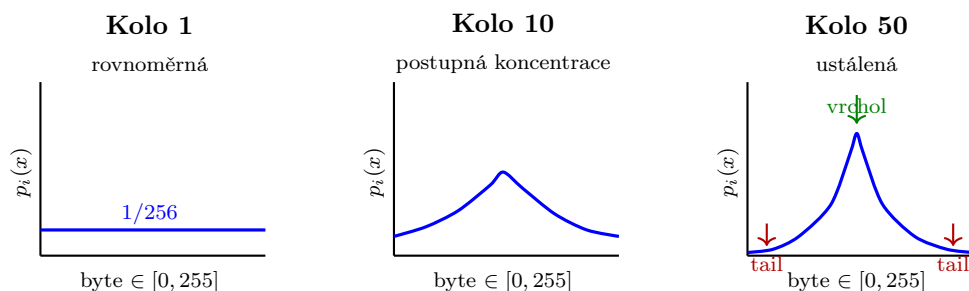
5.3 Operační rozsah $p_{\text{upd}} \in [0,6, 0,95]$

p_{upd} je pravděpodobnost, že se daný vektor v daném kole aktualizuje. V referenci v2 je $p_{\text{upd}} = 0,75$. Empirické sweepy ukazují velmi úzké okno funkcionality:

- $p_{\text{upd}} \leq 0,5$: drift se tvoří příliš pomalu. Po ~ 100 kolech zůstává $\mathcal{S}$ téměř rovnoměrná, není dost rozdílu mezi tail a center, tail-selekce nevybírá strukturně oddělené kandidáty $\Rightarrow$ **protokol nekonverguje**.
- $p_{\text{upd}} = 0,75$ (**sweet spot**): medián konvergence ≈ 49 kol, $\mathcal{S}$ entropie $\approx 7,55$ bit/byte, tail kandidáti $\log_2 \Pr \approx -160$.
- $p_{\text{upd}} \geq 0,95$: $\mathcal{S}$ kolabuje k delta (per-byte $\max p \rightarrow 1$), tail-selekce ztratí význam, kandidáti se koncentrují v "typické" oblasti $\Rightarrow$ **přímý sampling útočníkem se stane efektivní**.

Vynucený rozsah na úrovni configu

Hodnoty mimo $[0,6, 0,95]$ jsou aktivně odmítány při načítání konfigurace (`configs/reference.json`). Pokus o instanciaci s $p_{\text{upd}} = 1,0$ shodí program s explicitní chybou. Není možné *tiše* volit nebezpečnou hodnotu.



Obrázek 5: Vývoj Stat distribuce $\mathcal{S}$ na jedné pozici i přes kola. V kole 1 je rovnoměrná (každý byte stejně pravděpodobný). Po ~ 10 kolech se vlivem R_t a $p_{\text{upd}} = 0,75$ začíná koncentrovat. Po ~ 50 kolech je strukturní: ostré vrcholy ("typické" byty s vysokou Pr) a údolí ("tail" byty s nízkou Pr). Bob *záměrně* vybírá kandidáty z tailu, kde útočníkův přímý sampling téměř nikdy netrefí.

5.4 Vizualizace: vývoj Stat distribuce přes několik kol

5.5 Proč je toto strukturálně jiné než TPM

Tree Parity Machine (TPM, 2002) byl historický protokol používající dvě neuronové sítě, které postupně synchronizovaly své váhy přes "bidirectional learning rule". TPM byl prolomen geometrickým útokem (Klimov-Mityagin-Shamir, 2003): synchronizační zprávy unikaly informaci o gradientu, útočník mohl rekonstruovat váhy.

Pro tech-savvy: rozdíl od TPM

TPM update: každá strana posune své váhy ve směru, který *minimalizuje rozdíl* se zprávou druhé strany. Tento směr je signál pro útočníka.

Grata Cascade update: každá strana aplikuje *identickou* aktualizaci na základě veřejného R_t . Žádný signál mezi stranami; žádný gradient.

Empirická validace: B.4 AES-Oracle útočník (Sekce 7) provedl $2,2 \cdot 10^{10}$ dotazů, Cohen's $d = 0,009$ ($\rightarrow$ no measurable signal). Stejný typ útoku, který zlomil TPM, tady selhal.

To není formální důkaz, že Grata Cascade je odolná proti všem TPM-stylovým útokům. Je to *strukturální argument* (cesta úniku chybí) *plus* empirická validace (signál nedetekován i s velkým výpočetním rozpočtem). Otevřený problém `prob:dynamic-formal` v §9 navrhuje formální redukci.

6 AES bez paddingu jako obrana proti zaplavení

Co si odnést: AES bez paddingu odebírá útočníkovi validační orákulum — špatný klíč produkuje pseudonáhodný šum, ne explicitní chybu. To je strukturální argument: i útočník s extrémním poolem získá víc kandidátů, ne víc informace. Pro reference v2 znamená kumulativní bariéra $\lambda = 120$ bitů zhruba 2^{120} vyhodnocení hashe na nalezení jediného vektoru, který přitom zůstává jedním z 2^{136} navzájem nerozlišitelných.

Čtvrté designové rozhodnutí (§3.4) řekneme znovu, ale tentokrát s konkrétními čísly. Co se přesně stane, když útočník přijde s 10^{15} kandidátními vektory?

6.1 Per-round AES klíč

Vrátíme se ke kroku 2 z §4.3. Bob vybere $M = 8$ password kandidátů, lexikograficky je seřadí, a spočítá:

$$K_t = H(v_{\pi(1)} \| v_{\pi(2)} \| \dots \| v_{\pi(M)}).$$

Tímto klíčem AES-256-CBC *bez paddingu* zašifruje seed σ_t (32 B) a pošle Alici. Alice musí najít stejnou kombinaci kandidátů, aby spočítala stejné K_t a dešifrovala správně.

Lexikografické řazení je nezbytné: bez kanonického pořadí by Alice a Bob mohli mít stejnou množinu, ale v jiném pořadí, a odvodit by různé klíče.

6.2 NoPadding: žádné validační orákulum

Standardní AES-CBC používá padding (např. PKCS7), který doplňuje data na násobek bloku. Klíčový side-effect: pokud útočník dešifruje špatným klíčem, dostane neplatný padding $\Rightarrow$ exception. To je *validační orákulum*: útočník ví "ne, tenhle klíč nebyl správný."

Grata Cascade používá `PaddingMode.None`. To znamená:

- Vstupní velikost musí být násobek 16 B (proto se vektory dorovnávají, viz F8 commitnutí v repu).
- Dešifrování *vždy uspěje* syntakticky — vrátí L bytů pseudonáhodných dat.
- Útočník nemá žádný indikátor, jestli je dešifrovaný výsledek správný.

Rozdíl mezi "vrácením chyby" a "vrácením šumu"

S paddingem AES říká útočníkovi: "tenhle klíč není dobrý, zkus jiný". Bez paddingu AES říká: "tady máš 32 bytů. Které jsou ty pravé? Hádej." Útočník musí mít externí způsob, jak rozlišit pravé od šumu. Ten externí způsob v Grata Cascade neexistuje.

6.3 Útočník s velkým poolem: zaplavení místo identifikace

Tři fáze útočnickova filtrování. Než se pustíme do čísel, jeden žargon, kterým budeme pracovat dál v této i následující kapitole. Útočník při zkoumání transkripce postupně třídí své kandidátní vektory přes tři filtry — jak postupně dostává od protokolu nové hashe:

- **Stage-a** = vektor prošel *prvním* filtrem (jeho první 4 bajty SHA-256 odpovídají některému h_{AB} z transkripce $A \rightarrow B$).
- **Stage-b** = vektor prošel *prvními dvěma* filtry (h_{AB} z $A \rightarrow B$ i h_{BA} z $B \rightarrow A$). To už je vzácnější.
- **Stage-c** = vektor prošel *všemi třemi* filtry (přidáno h_P z Final zprávy). Pokud takový vektor existuje, je to skutečný password kandidát — nebo extrémně nepravděpodobná náhoda.

Číselně: každá fáze odřízne další $8 \cdot h$ bitů kandidátů, kde h je délka hashe. Pro reference v2 ($h_{AB} = 5$, $h_{BA} = 2$, $h_P = 8$ bajtů): stage-a redukuje o 40 bitů, stage-b o dalších 16, stage-c o dalších 64. Dohromady $\lambda = 120$ bitů. Cílem útočníka je dostat alespoň jednoho stage-c přeživšího.

Předpokládejme útočníka s P kandidátními vektory. Kolik z nich projde stage-b filtrem (shodují se s $A \rightarrow B$ i $B \rightarrow A$ hashi)?

$$E[\text{stage-b survivors}] = \frac{P \cdot N \cdot M}{2^{8(h_{AB} + h_{BA})}}.$$

Pro reference v2 ($N = 4096$, $M = 8$, $h_{AB} + h_{BA} = 7$ B = 56 bitů):

A teď klíčový moment: i když má útočník 10^{18} vektorů a najde $\approx 5 \cdot 10^5$ stage-b survivors, *nemá způsob, jak je rozlišit*. Každý prošel filtrem, každý dává po AES dešifrování pseudonáhodných 32 bytů. Bez validačního orákula útočník musí každého zkusit v dalších kolech (kde se mu signál také ztratí).

Pool útočníka P	Očekávání stage-b survivors
10^{12} (terabyte poolu)	≈ 1
10^{15} (petabyte poolu)	≈ 500
10^{18} (exabyte poolu)	$\approx 5 \cdot 10^5$

Tabulka 5: S rostoucím poolem útočníka roste počet stage-b survivors lineárně. Důležité: každý survivor je *nerozlišitelný* kandidát — útočník neví, který je správný. Růst poolu přidává šum, ne signál.

6.4 Proč krátký h_{BA} pomáhá obraně, ne útočníkovi

Zdá se kontraintuitivní, ale: kdybychom h_{BA} *prodloužili* z 2 B na 8 B, počet stage-b survivors by pro $P = 10^{15}$ klesl z ≈ 500 na zanedbatelnou hodnotu (řádu 10^{-12}). Útočník by dostal jednoznačného kandidáta, kterého by mohl ověřit dalšími koly.

Krátký h_{BA} záměrně udržuje stage-b survivors > 1 pro extrémní pooly $\Rightarrow$ útočník čelí *zaplavení* (mnoho kandidátů, žádný odlišný), ne *specifické identifikaci* (jeden kandidát, ověřitelný). Síla filtru je strukturálně přesunuta do h_{AB} a h_P , kde nepoškozuje flooding obranu.

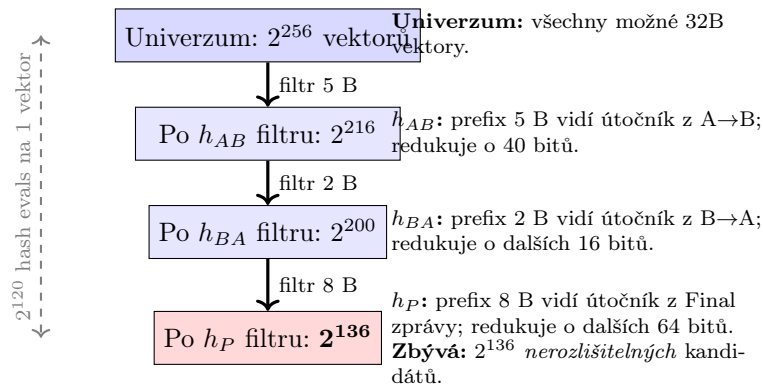
6.5 Cumulative preimage barrier a 2^{136}

Co *vidí* útočník o jednom konkrétním password vektoru v ? Tři hashe celkem:

- $H[0:5](v)$ z A $\rightarrow$ B fáze.
- $H[5:7](v)$ z B $\rightarrow$ A fáze.
- $H[7:15](v)$ z Final fáze.

Dohromady $5 + 2 + 8 = 15$ B = **120** bitů hashových omezení. Aby útočník našel *jakýkoli* vektor splňující tuto bariéru, potřebuje (za předpokladu náhodného orákula SHA-256) 2^{120} hashů — to je $1,3 \cdot 10^{36}$ operací. Přibližně tolik, kolik je atomů v miliardě lidských těl.

Ale i pokud útočník magickou silou jeden takový vektor najde, kandidátní množina nerozlišitelných má velikost $2^{8 \cdot 32 - 120} = 2^{136} \approx 8,7 \cdot 10^{40}$. Z těch 8,7 trilionu trilionu kandidátů je *právě jeden* správný. Bez validačního orákula útočník neví, který.



Obrázek 6: Kaskáda obran kolem jednoho password vektoru. Útočník vidí 3 hashové prefixy z různých fází protokolu ($5 + 2 + 8$ B = 15 B = 120 bitů). Najít jakýkoli vektor splňující všechny 3 vyžaduje 2^{120} hashů. Ale i takový vektor je 1 z 2^{136} nerozlišitelných kandidátů — bez validačního orákula nelze říct, který je ten správný.

6.6 Tři obrany dohromady

Na konec krátké shrnutí: tři strukturální obrany Grata Cascade, jak se vzájemně podporují:

- **Drift (Stat) + tail selekce** (§5): kandidáti jsou *strukturálně* v tailu, nedosažitelní přímým samplingem.
- **AES bez paddingu + krátký h_{BA}** (§6): kandidáti jsou *nerozlišitelní* při zaplavení, žádné validační orákulum.
- **Cumulative preimage barrier** ($\lambda = 120$ bitů): tvrdá dolní mez na útočnickou práci, nezávislá na drift/AES.

Útočník, který chce K^* , musí současně obejít *všechny tři*. Empirická evaluace (Kapitola 7) ukazuje, že se to nepodařilo žádné z 5 evaluovaných tříd útočníků na 300 transkriptech každý.

7 Empirické důkazy

Co si odnést (změřeno empiricky): 50 000 běhů reference v2 dalo 99,992% úspěch; výstupní klíče prošly statistickou baterií NIST SP 800-22 (12/12); pět tříd pasivních útočníků na 300 transkriptech každý nedosáhlo žádného obnovení klíče. Všechna čísla pocházejí z reálných spuštění protokolu, ne z teoretických odhadů. Details, intervaly spolehlivosti a kalibrační skeny jsou níže.

Doteď jsme mluvili o tom, *co protokol dělá a proč by měl fungovat*. Tato kapitola ukazuje *co se reálně stalo*, když jsme protokol pustili. Nejde o teoretické odhady ani simulace — jde o reálné běhy implementace v C# (.NET 9, post-F3 SHA-NI), reprodukovatelné z repu pod `configs/reference.json` a `reports/refv2/`.

Jak číst tabulky této kapitoly

V tabulkách budou opakovaně tři statistické pojmy. Zjednodušeně:

- **NIST SP 800-22** je standardní baterie 12 testů, kterými se ověřuje, jestli posloupnost bitů „vypadá jako náhodný šum“. Každý test buď projde (PASS) nebo selže pod nastavenou hladinou významnosti $\alpha = 0,01$. „12/12 PASS“ znamená, že po 1 000 klíčích ani jeden ze standardních testů nezachytil odchylku od náhodnosti.
- **Clopper-Pearsonův horní 95% interval (CP horní 95% CI)** je konzervativní statistický strop: máme k úspěchů z n pokusů a chceme říct „míra úspěchu v populaci s 95% jistotou nepřevyšuje toto číslo“. Pro 0/300 dává horní mez $\approx 1,22\%$ — čteme jako „ani s 95% jistotou neumíme tvrdit, že rate selhání je pod 1,22%“.
- **Cohenovo d** je velikost efektu: $d \approx 0$ znamená „žádný měřitelný rozdíl“, $d = 0,2$ malý efekt, $d = 0,5$ střední, $d = 0,8$ velký. Pro náš diskriminátor je $d = 0,009$ na hraně Poissonova šumu — stejně jako kdybychom srovnávali dvě sady losování mince.

Plná definice viz glosář v §10.

Klíčová čísla budeme citovat opakovaně, takže si je shrňme hned:

Empirický headline reference v2

- 50 000 konvergenčních běhů: 99,9920% úspěch (4 divergence). CP horní 95% CI: $2,05 \times 10^{-4}$.
- $h_{AB} = 6$ ablance (50 000 běhů): 0 **divergencí**. CP horní 95% CI: $7,38 \times 10^{-5}$, pod baseline.

- NIST SP 800-22: 12/12 PASS přes 1 000 klíčů, nejnižší p -hodnota 0,3156.
- Pět tříd pasivních útočníků (B.1, B.2, B.3, B.4, B.7), 0/300 obnovení K^* . CP horní per třída: 1,22%.
- B.4 AES-Oracle diskriminátor: Cohenovo $d = 0,009$ přes $2,05 \times 10^{10}$ dotazů — **žádná měřitelná síla**.

Nyní si projdeme jednotlivé experimenty a co znamenají.

7.1 Konvergence: 50 000 běhů, 4 divergence

Reference v2 byla testována na 5×10^4 nezávislých bězích (5 paralelních shardů po 10 000). Každý běh: pár Alice-Bob nezávisle inicializovaných, protokol běží do terminace, vyhodnotíme dvě binární metriky:

- **Final-hit (FH)**: dosáhli protokoly Final fáze (terminace τ trigger)?
- **Keys-match (KM)**: shoduje se K^* Alice s K^* Boba?

Výsledky jsou v Tabulce 6.

Výsledek	Počet běhů	Míra
FH = true $\wedge$ KM = true (úspěch)	49 996	99,9920%
FH = true $\wedge$ KM = false (divergence)	4	0,0080%
FH = false (strukturální selhání)	0	0,0000%

Tabulka 6: Konvergenční baseline reference v2. Per-shard distribuce divergencí $\{1, 0, 2, 0, 1\}$ je Poisson-konzistentní s mírou $\sim 8 \times 10^{-5}$. Distribuce iterací: medián 49, p95 62, max 90. Median elapsed čas: 1,71 s/běh.

Co znamenají divergence? Ze 4 případů (z 50 tisíc) Alice a Bob doběhli do terminace, ale jejich K^* se neshodly. To je strukturálně nezávažné — v takovém případě se na vyšší úrovni (transportní) detekuje neshoda klíčů a sezení se restartuje. Ale je to známka, že h_{AB} kolize stále občas projdou (analyticky $\sim 8 \times 10^{-5}$, pozorováno $4/50000 = 8 \times 10^{-5}$ — přesně ladí).

Jaký je horní strop? Clopper–Pearsonova jednostranná horní 95% CI na míru selhání: $2,048 \times 10^{-4}$. To je explicitní statistický strop — s pravděpodobností 95% míra v populaci nepřesahuje 2×10^{-4} .

7.2 Ablace $h_{AB} = 6$: jak posílit h_{AB} na nulové divergence

Reference v2 má $h_{AB} = 5$ B (40 bitů). Náš model predikuje, že kolize h_{AB} jsou dominantním zdrojem reziduálních divergencí: 4 divergence z 50 tisíc, ladící s analytickým $\mathbb{E}[\text{kolize}] = N^2/2^{40} = 4096^2/2^{40} \approx 1,5 \times 10^{-5}$ na kolo, $\times 50$ kol/sezení $\approx 7,5 \times 10^{-4}$, kombinováno s $1/M = 1/8$ pravděpodobností "trefit" password slot $\approx 9 \times 10^{-5}$ — v rozumné shodě s pozorovanými 8×10^{-5} .

Pak: zvýšení h_{AB} z 5 na 6 (přidání 8 bitů) by mělo míru kolizí snížit 256 $\times$, na $\sim 3 \times 10^{-7}$. Empirický test:

Profil **reference_h_AB6** (klon reference v2 s $h_{AB} = 5 \rightarrow 6$, ostatní stejné) byl spuštěn na 5×10^4 konvergenčních běhů.

Empirické potvrzení modelu

$h_{AB} = 6$ ablace ukazuje předpovědi:

- Ano, kolize h_{AB} byly dominantní zdroj. +1 byte je odstranil.

Profil	h_{AB} (bit)	N	n	Pozorováno	CP horní 95% CI
F13-v2 IoT	32	1024	200	$1/200 = 5 \times 10^{-3}$	$2,8 \times 10^{-2}$
Reference v2 (kotva)	40	4096	50 000	$4/50\,000 = 8 \times 10^{-5}$	$2,05 \times 10^{-4}$
$h_{AB} = 6$ ablance	48	4096	50 000	0/50 000	$7,38 \times 10^{-5}$
F11-v2 pq128	48	8192	200	0/200	$1,5 \times 10^{-2}$

Tabulka 7: Škálování h_{AB} rezidual napříč 4 empirickými body (16 bitů h_{AB} rozpětí: 32 $\rightarrow$ 48). Řádek $h_{AB} = 6$ je nejčistší test — variuje pouze h_{AB} vůči baseline, ostatní parametry drženy. Pozorované 0 divergencí na 50 tisíc běhů, CP horní 95% CI **leží pod** reference v2 baseline.

- Predikované $\sim 3 \times 10^{-7}$ míry; pozorovaná horní mez $7,38 \times 10^{-5}$ (50k není dost na rozlišení od nuly, ale je pod baseline).
- Konvergence se nezpomalila: medián iterací pořád 49 (= reference). Navýšení objemu dat: +4 KB/kolo při $N = 4096$ ($\sim 20\%$ kanálu A $\rightarrow$ B).

Tohle je důvod, proč se **reference_h_AB6** kvalifikuje jako **kandidát na v3 reference profil**.

7.3 NIST SP 800-22: vypadají klíče jako náhodný šum?

Sebrali jsme 1 000 nezávislých klíčů reference v2 (999 použitelných po vyloučení 1 KM-divergence) a aplikovali na ně 12-test sadu NIST SP 800-22 — standardní statistickou baterii pro pseudonáhodné generátory. Agregovaný bitstream: 255 744 bitů.

Výsledek: 12/12 PASS na $\alpha = 0,01$. Nejnížší p -hodnota 0,3156 (Binary Matrix Rank — nejtěžší test, ale s velkou rezervou). To znamená: K^* klíče Grata Cascade jsou statisticky *nerozznatelné od skutečně náhodných sekvencí* dle nejširší standardní baterie.

7.4 Kalibrační sweepy: kde to přestává fungovat

V Kapitole 5 jsme tvrdili, že $p_{\text{upd}} \in [0,6, 0,95]$ je úzký operační rozsah. Tady máme empirickou validaci přes 3 sweepy.

Sweep p_{upd} . Hodnoty $\{0,1, 0,25, 0,5, 0,75, 0,9, 0,95, 1,0\}$ s $\theta_{\text{cand}} = -95$ (zapnut filtr):

p_{upd}	FH/N	iter(p50)	Stat entropie	Stat max prob	avg pw log ₂
0,10	0/50	201 (cap)	7,55	0,04	—
0,25	0/50	201 (cap)	7,09	0,09	—
0,50	0/50	201 (cap)	6,13	0,16	—
0,75 (ref)	200/200	67	4,49	0,33	-165,10
0,90	174/200	106	2,69	0,58	-167,45
0,95	0/50	201 (cap)	1,68	0,74	—
1,00	0/50	201 (cap)	0,00	1,00	—

Tabulka 8: p_{upd} sweep validuje úzký operační rozsah. Pod 0,5: drift se netvoří (Stat zůstává blízko rovnoměrné, entropie 7,55), tail-selekce je neefektivní. Nad 0,95: Stat kolabuje (max-prob roste k 1), tail-selekce ztrácí strukturu. Sweet spot: 0,75.

Sweep s (perturbace seedu). Otestovali jsme $s \in \{1, 2, 3, 5, 7, 10, 15, 30\}$. **Konvergence je invariantní** přes celý rozsah: 1600/1600 úspěchů, medián iter rozptyl ± 1 , avg pw log₂ rozptyl

$\pm 1,3$ bitů. To je důležité: hustota poolu ρ_{pool} se mění o 182 bitů (od $-38,7$ při $s = 1$ k $-220,9$ při $s = 30$), aniž by to ovlivnilo konvergenci. Drift dominuje hustotě poolu.

Při tom jsme provedli i **re-test B.4 AES-Oracle útočníka** při hustých podmínkách ($s = 1$, $\rho \approx 2^{-38}$ — 39 bitů hustší než reference): nulový diskriminační signál se potvrdil přes $2,3 \times 10^8$ near-seed vzorků. To empiricky vyvrací hypotézu, že hustota poolu je obrana — skutečnou obranou je AES derivace klíče bez paddingu.

Ablace θ_{cand} . Zopakovali jsme p_{upd} sweep s vypnutým CandMax filtrem:

p_{upd}	FH/N (filtr -95)	FH/N (vypnuto)	avg pw $\log_2$ vypnuto
0,75	200/200	200/200	-158
0,90	174/200	200/200	-150
0,95	0/50	200/200	-143
1,00	0/50	200/200	-53

Tabulka 9: Ablace bezpečnostní role θ_{cand} : bez filtru vysoké p_{upd} zachovává konvergenci, ale kandidáti kolabují z $\log_2 \text{Pr} \approx -160$ na -53 při $p_{\text{upd}} = 1,0$. Ti jsou pak v rekonstruovatelné Stat distribuci a útočník je může přímo samplovat. Filtr je tedy strukturální obrana, ne jen "cleanup".

7.5 Per-profile validace: funguje to i mimo referenci?

Validovali jsme všech 5 profilů samostatně (200 běhů každý + 100 klíčů NIST baterie):

Profil	FH/N	KM/N	iter(med)	NIST PASS/N/A	Poznámka
Reference v2	200/200	200/200	49	12/12, 0 N/A	Primární
IoT	200/200	200/200	68	11/12, 1 N/A	$h_P = 4$ po fixu
Mobile	200/200	200/200	48	11/12, 1 N/A	—
HighSec Classical	200/200	200/200	89	11/12, 1 N/A	—
HighSec PQ128	50/50	50/50	69	10/12, 1 N/A	1 marginal Block Freq

Tabulka 10: Per-profile validace přes 5 profilů. "N/A" je Binary Matrix Rank přeskočený (100-key baterie dává 25 600 bitů, BMR potřebuje $\geq 38 912$ bitů). HighSec PQ128 s 1 marginálním selháním (Block Frequency, $p = 0,0035$) konzistentní s šumem multi-test baterie; disambiguation re-run plánovaný.

Lekce z IoT: původní IoT konfig ($h_P = 3$ B = 24 bitů preimage) vykazoval $\sim 2,5\%$ silent key mismatch (tichá kolize Final hashe). Zvýšení na $h_P = 4$ B (32 bitů) oprava: 200/200 KM, +8 bitů kumulativní bezpečnosti, +8 B/Final zpráva. Ukazuje, že per-profile kalibrace není luxus — různé scale potřebují různé hashy.

7.6 Pět tříd pasivních útočníků

Implementovali jsme a evaluovali 5 tříd pasivních útočníků. První 4 (B.1–B.4) na referenci v2 přes $N = 300$ transkriptů; pátý (B.7) na research-only **break_demo** (úmyslně oslabená konfig pro pedagogickou demo).

B.1 Baseline Random. Rovnoměrný náhodný sampling, hledá shodu hashů. Time budget 600 s/transkript. Očekávaná práce na slot: 2^{64} .

B.2 Hash-First Filtered. Kaskádová filtrace přes 3 hashe (A→B 5B, B→A 2B, Final 8B). Filtrační faktor 2^{120} . Pool 10^7 .

B.3 Tail-Informed Sampler. Modifikace B.2, kde generuje kandidáty z rekonstruovaného Stat tailu. Pool 10^7 .

B.4 AES-Oracle Discriminator. Po hypotetickém stage-b kandidátovi dešifruje AES, regeneruje near-seed pool, validuje proti hashům dalšího kola. Měří diskriminační sílu (TP/FP rates, Cohenovo d). $K = 10^6$ vzorků na round-pair.

B.7 Stat-Sorted Enumeration. Místo náhodného samplingu B.7 vektory *deterministicky řadí* podle jejich pravděpodobnosti pod $\mathcal{S}$ a postupně bere vektory v tom pořadí (od nejvíce po nejméně pravděpodobné, nebo opačně). Implementačně to dělá pomocí prioritní fronty — cílí přesně na vrcholy nebo na škvíry kola štěstí, ne na náhodný sektor. Dva módy:

- *Typical*: vektory v klesající Stat pravděpodobnosti (typické bajty); cílí na Alicin pool.
- *Tail*: vektory v rostoucí Stat pravděpodobnosti (vzácné bajty); cílí na protokolovou CandMax-filtrovanou password sadu.

Validace na **break_demo** ($\lambda_{\text{preimage}} = 56$ bitů, parametry úmyslně oslabené tak, aby byl útok teoreticky proveditelný, ale stále nepatrně nad prahem prolomení): 0/300 úspěšných obnovení klíče v obou módech. Dual-mode chování ukazuje, že kaskáda blokuje enumeraci podle Stat distribuce v *obou* směrech.

Útočník	Běhů	Recovery K^*	Úspěšnost	95% CI horní	Avg time/run
B.1 Baseline Random	300	0	0,000%	1,222%	600,0 s (cap)
B.2 Hash-First	300	0	0,000%	1,222%	56,7 s
B.3 Tail Sampler	300	0	0,000%	1,222%	130,5 s
B.4 AES-Oracle	300	0	0,000%	1,222%	335,1 s
B.7 Stat-Sort. Tail (na break_demo)	300	0	0,000%	1,222%	59,0 s (10^7 pool)

Tabulka 11: Evaluace pasivních útočníků. B.1–B.4 pod reference v2 přes $N = 300$ transkriptů (sdílená sada). Celkový výpočet $\sim 10^{11}$ SHA-256 vyhodnocení, reálný čas 6 h 33 min na 22 paralelních pracovnících. B.7 reportuje **break_demo** dual-mode (Tail při 10^7 pool); evaluace B.7 proti reference v2 odložena.

Hlavní výsledky. **B.4 Cohenovo $d = 0,009$:** přes 10 262 round-pairs a $2,05 \times 10^{10}$ dotazů, mean TP 0,003411, mean FP 0,002923. Empirická FP je v predikčním pásmu SHA-orákula ($\mathbb{E}[\text{FP}] = 0,003725$). **Žádný měřitelný diskriminační signál.**

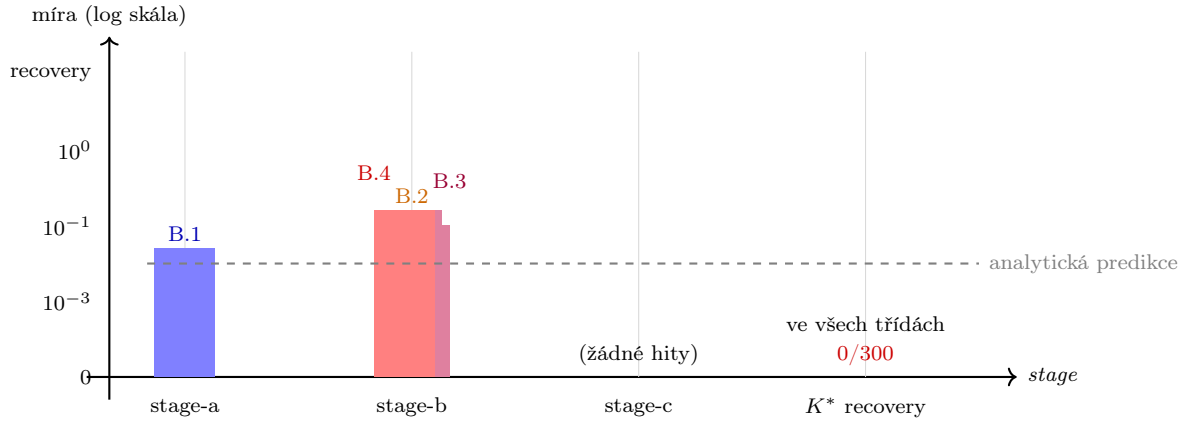
Validace stage kaskády (B.2): přes 300 transkriptů, 503 kol se stage-a hity (515 celkem), 0 stage-b, 0 stage-c. Hity stage-a odpovídají analytické kaskádě v rámci Poisson šumu. Filtr funguje jak má.

7.7 Vizualizace: efektivnost útočníků vs. random baseline

7.8 Co celé experimenty říkají dohromady

Souhrn empirické evaluace

- Protokol **téměř vždy konverguje** (99,992% nebo lépe na 50k bězích).
- Výstupní klíče vypadají **statisticky náhodně** (NIST 12/12 PASS).
- Operační rozsahy parametrů jsou **empiricky vymezené** a vynucované při configu.
- Žádná z 5 tříd útočníků (klasifikované strategie + budget $\sim 10^{11}$ SHA evals) **neobnovila**



Obrázek 7: Schematický pohled na účinnost útočníků (B.1–B.4) napříč kaskádou. Stage-a: všichni útočníci dostávají hity konzistentní s analytickou predikcí (Poisson šum kolem dashed čáry). Stage-b: 0 hitů — kombinovaný $h_{AB} + h_{BA}$ filtr (56 bitů) zastaví všechny. Stage-c a K^* recovery: nulové. Žádný útočník neobnovil K^* z 300 transkriptů.

K^* .

- B.4 AES-Oracle, nejsofistikovanější discriminator-based útok, má **Cohenovo** $d = 0,009$ — vize TPM-stylového gradient leaku tady neselhal v "porazení"— selhal ve "vůbec měřitelnosti".
- $h_{AB} = 6$ ablance ukazuje, že posílení λ na 128 bitů je **bez pozorovaného dopadu na konvergenci** (medián iter beze změny, +4 KB/kolo objemu dat).

To není formální důkaz, že protokol je nezdolatelný. Je to silná empirická evidence, že *v rámci testovaného rozsahu* se chová podle slibu. Co zůstává otevřené (ML útočníci, multi-round agregace, formální redukce) si projdeme v Kapitole 9.

8 Srovnání s jednorázovými KE primitivami

Co si odnést: Grata Cascade je v objemu přenášených dat 3 řády horší než ML-KEM (postkvantový standard) a téměř 5 řádů horší než ECDH (klasický standard). Její potenciální přínos není efektivita, ale strukturální kryptografická diverzifikace: stojí na třetí nezávislé bázi (hash pre-image + drift), takže v hybridním X-Wing-style stacku přidává třetí rovnocennou vrstvu, jejíž souběžné prolomení s DLP i mřížemi by vyžadovalo tři nezávislé pokroky kryptanalýzy.

Grata Cascade je v této knížce a v paperu pozicována jako *jednorázová* primitiva domluvy klíče. Srovnání tedy směřujeme k existujícím jednorázovým KE primitivám (ECDH, ML-KEM, X-Wing hybrid, Merckovy puzzly), ne ke continuous key agreement (CKA) konstrukcím jako Signal Double Ratchet, MLS nebo Triple Ratchet — vztah ke CKA je popsán v Kapitole 9 (`prob:cka-extension`).

Grata Cascade je v objemu přenášených dat *přibližně tři řády horší* než ML-KEM (1,2 KB) a téměř pět řádů horší než ECDH (32 B). To je zásadní inženýrská nevýhoda a vylučuje ji z drtivé většiny produkčních deploymentů, kde je objem dat citlivý parametr (mobilní data, IoT). Optimalizační cesty jsou otevřeny v `prob:bandwidth` (Kapitola 9).

Tak proč by ji měl někdo chtít?

Schéma	Báze	PQ bezpečnost	Objem dat / sezení	Formální důkaz
ECDH (X25519)	DLP	Ne (Shor)	~ 32 B	Ano (ROM)
ML-KEM-768	Module-LWE	Ano	$\sim 1,2$ KB	Ano (ROM)
X-Wing	DLP + Module-LWE	Ano	$\sim 1,3$ KB	Ano
Merklovy puzzly (1978)	Hash preimage	Ano (Grover)	$O(N^2)$ work	Ano (IR89)
Grata Cascade ref v2	Hash preimage + drift	Ano (konj.)	~ 1 MB	Ne (otev.)

Tabulka 12: Srovnání jednorázových KE primitiv. „Konj.“ = konjekturální (strukturálně argumentováno, formálně neprokázáno). „IR89“ = Impagliazzo–Rudich black-box bound. Objem dat pro ECDH a ML-KEM je round-trip; pro Grata Cascade celé sezení ($49 \text{ kol} \times 20 \text{ KB}$).

8.1 Hodnota: třetí nezávislá vrstva v hybridní kompozici

Současný NIST-doporučený hybrid X-Wing kombinuje X25519 (ECDH) s ML-KEM-768. Útočník musí prolomit *obě* báze pro získání klíče:

- **DLP** prolomí kvantový počítač (Shor, očekávané). Pak X25519 padá, ale ML-KEM drží.
- **Module-LWE** prolomí neočekávaný útok. Mřížová kryptografie je relativně mladá (NIST standardizace 2024); pokud někdo najde polynomiální algoritmus, ML-KEM padá. Pak X25519 drží (před kvantem), ale ne před útočníkem s kvantovým počítačem.
- **Obě současně** (kvantový počítač + mřížový průlom): X-Wing padá. Žádná pojistka.

Grata Cascade může v hybridním stacku přidat **třetí nezávislou vrstvu**, která stojí na úplně jiném principu než DLP nebo mříže:

- **Báze: hash preimage + statistický drift.** Není závislá na DLP ani na mřížích. Hash funkce je strukturálně jiný typ kryptografické primitivy než algebraické konstrukce.
- **Dlouhá prověřenost.** SHA-256 existuje od roku 2001 a je běžně nasazená v prakticky všech kryptografických kontextech (TLS, signatures, code commitments). Útoky proti hashům jsou intenzivně zkoumány desítky let.
- **Postkvantová bezpečnost je solidní.** Groverův algoritmus zrychluje preimage o $\sqrt{\cdot}$, takže pro SHA-256 plný hash zůstává postkvantově bezpečný na 2^{128} . Naše kumulativní bariéra $\lambda = 120$ bitů je tedy postkvantově 2^{60} — na hraně slabšího prahu; proto v3 kandidát **reference_h_AB6** posiluje na $\lambda = 128$ bitů, post-Grover 2^{64} .

Diverzifikace, ne nahrazení

Grata Cascade *není* pokus nahradit ECDH ani ML-KEM. Ty jsou výrazně efektivnější (řádově $1\,000\times$ až $100\,000\times$ menší objem dat) a mají formální důkazy. Cíl Grata Cascade: být *třetí rovnocennou vrstvou* v hybridním stacku — konkrétně v rozšíření X-Wing-style: $X25519 \oplus \text{ML-KEM-768} \oplus \text{Grata Cascade}$. Pokud útočník prolomí dvě z těchto tří bází (např. kvantový počítač + mřížový průlom), Grata Cascade jako třetí drží. Pokud by Grata Cascade padla, X-Wing pořád drží před tím útočníkem. Hodnota je v *nezávislosti* bází, ne ve výkonu.

8.2 Vůči Merklovým puzzlům

Klasická hashová KE primitiva, Merklovy puzzly (1978), dosahuje $O(N^2)$ útočnické práce nad $O(N)$ legitimní práce. To je dle Impagliazzo–Rudich (1989) optimum v black-box modelu: žádná konstrukce čistě nad jednosměrnou funkcí nedosáhne víc než kvadratického zabezpečení. Pro

$\lambda = 128$ bitů to vyžaduje $N \approx 2^{64}$ rebusů, což je deployment-prohibitivní bandwidth (řádově exabajty).

Grata Cascade konjekturálně překračuje tento black-box bound díky kombinaci hashe se statistickým driftem. Pokud bude formálně potvrzeno (otevřený problém `prob:dynamic-formal`), bude to první nasaditelná hash-only KE primitiva s lepším bandwidth-bezpečnost tradeoffem než Merkle. Ale i bez formální redukce je už dnes Grata Cascade 3 řády lepší než Merkle ve $\lambda = 120$ -bit režimu (1 MB sezení vs. exabajty).

8.3 Co Grata Cascade postrádá

Buďme upřímní: kromě objemu dat má protokol další slabiny.

- **Žádný formální důkaz.** ECDH a ML-KEM mají *game-based* bezpečnostní redukce v modelu náhodného orákula (ROM) na obtížnost svých underlying problémů; redukce jsou peer-reviewed a standardizované. Grata Cascade má naproti tomu jen *strukturální argumenty* a *empirickou validaci* pro pasivního útočníka. Otevřený problém `prob:dynamic-formal` (§9) navrhuje analogickou redukci, ale není hotová — díra mezi „strukturálně přesvědčivé“ a „formálně dokázané“ zůstává.
- **Vyšší výpočetní náročnost.** $\sim 1,71$ s na konvergenci na desktop CPU vs. milisekundy na ECDH update. Pro IoT/baterii má vliv.
- **Plně neevaluované parametrické profily.** Mobile a HighSec jsou kalibrované, ale nemají per-profile útočnickou evaluaci s $N = 300$ transkripty (otevřený problém `prob:parametric`).
- **Není CKA.** Grata Cascade produkuje *jeden* klíč K^* na sezení; nepokrývá průběžnou rotaci mezi message-i, kterou v Signal stacku zajišťuje Triple Ratchet. Rozšíření na CKA primitivu je samostatný open direction (`prob:cka-extension`).
- **Nestabilní vůči aktivním útočníkům.** Bez autentizovaného transportu (TLS, Noise) může aktivní útočník modifikovat R_t a degradovat protokol. Tři volitelné obrany (statistická detekce, deterministické R_t , veřejný R -pool) jsou v paperu v9 v Dodatku, ale netestovány v širokém nasazení.

Nehrabeme to pod koberec — v Kapitole 9 si tyto otevřené problémy projdeme detailněji.

9 Co ještě nevíme: otevřené problémy

Co si odnést: dva zásadní otevřené problémy: `prob:cka-extension` (rozšíření jednorázové primitivy na CKA s formálními FS/PCS zárukami; primárně cesta k third-layer ratchet) a `prob:dynamic-formal` (formální redukce dynamické asymetrie na SHA-256 preimage hardness). Dále jsou otevřené ML útočníci, multi-round agregace slabých signálů, optimalizace objemu dat a flooding stress test. Naopak „ h_{AB} reziduální divergence“ a „Break-Demo evaluace“ byly v8 empiricky uzavřeny.

Žádný výzkum není hotový. Tato kapitola je upřímný seznam otázek, na které ještě nemáme odpověď. Některé jsou *zásadní* (řešení by významně změnilo náš pohled), jiné jsou *inženýrské* (jejich řešení by udělalo protokol praktičtější), jiné jsou *rozšiřující* (zajímavé, ale ne nutné pro hlavní tvrzení).

9.1 Centrální problémy

`prob:cka-extension` — Rozšíření na CKA primitivu. Grata Cascade v této práci je *jednorázová* primitiva: jedno sezení, jeden klíč K^* , hotovo. Pro plnou integraci do messaging stacku ve stylu Signal je potřeba *průběžná domluva klíče* (CKA): protokol, který produkuje sekvenci

klíčů $\{k_1, k_2, k_3, \dots\}$ napříč konverzací s formálními zárukami forward secrecy a post-compromise security. Načrtáváme tři možné cesty:

- **Path A — Repeated-session chaining (primární cíl).** Spustit GC opakovaně, kde každé nové sezení použije předchozí K^* jako seed plus čerstvou náhodnost. Drahé (~ 1 MB per rotace), ale čistá redukce na bezpečnost jednoho sezení. Hlavní target pro follow-up paper s formální redukcí.
- **Path B — Symetrický ratchet nad GC bootstrap.** Použít GC k odvození K_0^* , pak $K_i^* = \text{HKDF}(K_{i-1}^*, \text{salt}_i)$ jako v Signal symetrickém KDF ratchetu. Levné a hned nasaditelné, ale PCS dědí jen z bootstrappingu, ne z ratchetu samotného.
- **Path C — Nativní CKA konstrukce.** Přeprogramovat GC tak, aby každé kolo emitovalo *vydaný* klíč plus *udržovaný* stav (drift pokračuje). Nejvíce novel, ale fundamental redesign + nová bezpečnostní analýza — long-term moonshot.

Proč je to zásadní: bez CKA varianty Grata Cascade nepokrývá use case „jeden klíč na zprávu“ (Signal). Path A je nejdefensibilnější následující paper a logická pokračovací cesta po vyřešení `prob:dynamic-formal`.

prob:dynamic-formal — Formalizace dynamické asymetrie. Statická asymetrie $R \approx n/k$ (Kapitola 2) dává *dolní mez* útočnické nevýhody pro jediné sezení. Dynamické mechanismy (drift, hash addressing, multi-round zpřesňování) tuto mez zesilují — ale o kolik a za jakých podmínek? Cíl: odvodit analytickou mez na útočnickou výhodu jako funkci (počet kol, parametry, Stat koncentrace) za předpokladu náhodného orákula pro SHA-256. Redukce na obtížnost preimage SHA-256 je podkomponentou.

Proč je to zásadní: jeho řešení by převedlo empirické bezpečnostní tvrzení ("nikdo to neprolomil přes 300 transkriptů") na formální kryptografickou záruku ("útočnická výhoda je $\leq 2^{-\lambda}$ pro ..."). Bez toho je Grata Cascade strukturálně přesvědčivá, ale nemá tvrdou matematickou kotvu pro single-session bezpečnost. Jeho řešení by zároveň poskytlo bázi pro Path A v `prob:cka-extension`.

9.2 Empirické rozšíření

prob:parametric — Plná parametrická útočnická analýza. Tři produkční profily (reference, Mobile, HighSec) jsou kalibrované, ale jen reference v2 má per-profile útočnickou evaluaci přes 5 tříd. Doplnit pro Mobile a HighSec přes $N \geq 300$ transkriptů. Také: konstrukce Pareto hranice v prostoru (bezpečnost, objem dat, konvergence, paměť).

prob:multiround — Multi-round agregace slabých signálů. Per-round B.4 diskriminátor má detekční sílu $d = 0,009$, neměřitelná. Útočník agregující přes k kol získává efekt $d_0\sqrt{k}$. Pro $d_0 = 0,1$, $k = 50$: $d \approx 0,7$, detekovatelné. Otázka: je takové agregace dosažitelná pro útočníky proti Grata Cascade — vzhledem k tomu, že single-round signál je pod prahem detekce?

prob:nist-strong — Posílená NIST evaluace na 10^6 klíčů. Současná NIST baterie používá 1 000 klíčů. Při $\geq 10^6$ klíčů s analýzou druhého řádu (chi-square test rovnoměrnosti na výsledné distribuci p -hodnot) by tvrzení statistické uniformity mělo vyšší spolehlivost. Proveditelné v rámci současného uspořádání, ~ 1 den výpočtu.

9.3 Nové třídy útočníků

prob:ml — ML útočníci. Natrénovat hluboký neuronový model na mapování (transkripce $\rightarrow$ bity K^*). Lepší než 50%? Pokud ano, kvantifikovat únik a identifikovat, které transkripční prvky nesou signál. Strukturálně odlišná třída od B.1–B.7 a není adresovaná současnou empirikou.

9.4 Inženýrská optimalizace

prob:bandwidth — Optimalizace objemu dat. Reference vyžaduje ~ 20 KB/kolo, dominováno N hash prefixy $A \rightarrow B$. Mohou Bloom filtry, dělení do bloků nebo amortizace přes kola snížit pod 5 KB při zachování bezpečnosti? Primárně inženýrská práce, ale s velkým dopadem na životaschopnost nasazení.

prob:flood-stress — Stress test obrany proti zaplavení při velkých poolech. Současná baterie útočníků (§7.6) evaluuje B.1–B.4 při $P = 10^7$. Při této velikosti poolu je očekávaný počet stage-b survivorů $\approx 4,5 \cdot 10^{-6}$ na kolo (vzorec $E = P \cdot N \cdot M/2^{56}$). Výsledek 0/300 tedy spíš znamená „*nikdo nedošel ani do stage-b*“ než „*stage-b zafungoval jako filtr proti reálnému zaplavení*“. Pro empirickou validaci AES NoPadding obrany pod skutečnou zátěží potřebujeme $P \geq 10^{10}$:

- **Predikce při $P = 10^{12}$:** ≈ 22 stage-b survivorů na sezení (50 kol), $\approx 1,2 \cdot 10^{-18}$ stage-c survivorů, ≈ 0 obnovení K^* . Měření by validovalo flooding obranu pod zátěží: pozorovaný počet stage-b/stage-c hitů by se mohl porovnat s analytickou predikcí, a zejména by se ověřilo, že přes desítky stage-b kandidátů na sezení *žádný* nepřežije do stage-c (nebo jen v souladu s Poissonovým šumem).
- **Paměťové nároky:** konstantní. Pool se streamuje (negeneruje se najednou), drží se jen $\sim$ KB hashů z transkripce. 16 GB GPU RAM je více než dost.
- **Časové nároky:**
 - B.2 (čisté SHA filtrování): současné $P = 10^7$ trvá ≈ 57 s/sezení. Lineární škálování dává při $P = 10^{12}$ na 22-jádrovém CPU ≈ 900 dní pro 300 sezení — prohibitivní. Na GPU (RTX 4060/4090: 5–10 GH/s SHA-256) se to redukuje na **8–16 hodin pro celý batch 300 sezení**.
 - B.4 (AES Oracle): AES decrypt na GPU má řádově horší propustnost než SHA-256, takže $P = 10^{12} \times 300$ je i na GPU dlouhodobý běh (dny až týdny). Praktická redukce: $N = 30$ sezení nebo $P = 10^{10}$.
- **Plán:** po obdržení dedikované RTX 4060 (cca 2026-05-13) spustit B.2 @ $P \in \{10^{10}, 10^{12}\} \times 300$ sezení a B.4 @ $P = 10^{10} \times 30$ sezení. Doplnit Tabulku 11 o nový sloupec/řádky a porovnat s analytickou predikcí.

prob:pupd-dynamic — Dynamický p_{upd} . Statický $p_{\text{upd}} = 0,75$ je empiricky kalibrovaný optim. Dynamická varianta, kde p_{upd} je odvozeno ze sdíleného tajného stavu, by vynutila útočníka rekonstruovat plnou Stat rodinu $\mathcal{S}_\theta$ pro neznámý θ , potenciálně přidávaje bity efektivní bezpečnosti. Bezpečnostní zisk a implikace pro konvergenci otevřené.

9.5 Výzkumné parametrické rodiny

prob:iot — IoT parametrická rodina. Plná kalibrace low-resource profilu ($L = 16$, $N \sim 1024$). Velikost nerozlišitelné množiny při těchto parametrech je pouze 2^{48} — v dosahu brute-force enumerace na moderních GPU. Plná kalibrace podle tří-rationale designu a útočnická evaluace otevřená.

prob:pq128 — 128-bit postkvantová rodina. Návrh a empirická validace profilu dosahujícího 128-bitové PQ bezpečnosti za Groverova zrychlení. Klíčová designová otázka: $\lambda_{\text{preimage}} \geq 256$ jako konzervativní cesta s velkým objemem dat, nebo strukturální argumenty kombinující velikost nerozlišitelné množiny + Stat korelaci (eleganternější, ale závislé na řešení **prob:dynamic-formal**).

9.6 Co je naopak uzavřeno

prob:hab-residual — Eliminace reziduálních divergencí h_{AB} [uzavřeno v v8]. Otevřený problém z dřívějších verzí paperu. Uzavřen v v8 přímou ablací: $h_{AB} = 6$ na referenční škále přes 5×10^4 běhů dalo 0 KM divergencí, CP horní 95% CI $7,38 \times 10^{-5}$, pod baseline reference v2 $8,0 \times 10^{-5}$. Plná zpráva v §7.2.

prob:breakdemo — Break-Demo parametrická podrodina [empiricky validováno v v8]. Otevřený problém z dřívějších verzí. V v8 redukováno na pedagogickou demonstraci: parametry $L = 8, N = 256, h_{AB} = 3, \lambda_{\text{preimage}} = 56$ bitů — úmyslně oslabené tak, aby útok byl teoreticky blízko proveditelnosti, ale stále nepatrně nad prahem prolomení. Empirický pilot pod baterií 5 tříd útočníků: 0/300 napříč všemi. Zjištěno, že kaskádový filtr blokuje enumeraci podle Stat distribuce v obou směrech — otevřená otázka je dosažitelná pouze při $\lambda \leq 48$ nebo s útočnickými třídami mimo rámec kaskádového filtru.

Honest about scope

Sedm otevřených problémů není známka, že protokol "není hotový". Je známka, že *se ptáme poctivě*: kdykoli máme nějaké tvrzení, máme i otázku, jak ho zlepšit nebo vyvrátit. Tahle hustota otevřených otázek je sama o sobě bezpečnostní vlastnost — je to průhledná pozice "tohle nevíme", ne "všechno víme".

10 Glosář a kde dál

10.1 Slovník pojmů

Klíčové termíny v populárním překladu. Pro plnou technickou definici viz `docs/glossary.md` (v repu) nebo paper v8 main.

CKA (Continuous Key Agreement)

Kontinuální domluva klíče. Mechanismus v messagingu, kde se klíče postupně mění (jak komunikace pokračuje), aby kompromitace jednoho stavu neohrozila ostatní.

Ratchet

V kryptografii bezpečného messagingu mechanismus, který *postupně přepíná klíč dopředu* — *nikdy zpět* (analogie západky u řehačky či ráčny). Útočník, který v jednu chvíli získá aktuální klíč, nemůže s ním rozšifrovat staré zprávy ani odvodit budoucí. *Triple Ratchet* = tři nezávislé ratchety vrstvené v Signalu (DH + KEM + symetrický KDF). Grata Cascade v současné podobě *není* ratchet (produkuje jeden klíč na sezení); rozšíření na CKA primitivu se sekvencí klíčů viz Kapitola 9, `prob:cka-extension`.

Forward Secrecy (FS)

Vlastnost: dnešní kompromitace klíče nedovoluje rozšifrovat *minulé* zprávy. (Kdyby vás zítra prozradili, dnešní zpráva je v bezpečí.)

Post-Compromise Security (PCS)

Vlastnost: po kompromitaci se protokol postupně "zahojí" — nové klíče vygenerované po incidentu jsou bezpečné, i když útočník dočasně viděl interní stav.

Diffieho-Hellman (DH, ECDH)

Klasický key exchange, g^x a g^y se vyměňují, sdílený klíč g^{xy} . Prolomený Shorovým algoritmem na kvantovém počítači.

ML-KEM

Module-Lattice based Key Encapsulation Mechanism. Postkvantový key exchange standardizovaný NIST 2024. Triple Ratchet ho používá jako druhou vrstvu.

KEM

Key Encapsulation Mechanism. Obecnější třída postkvantových primitiv, kam patří ML-KEM (jakoby asymetrické šifrování pro key exchange).

SHA-256

Standardní hash funkce. Jednosměrná: ze vstupu hash snadno, obráceně to prakticky nelze. 2^{128} post-Grover bezpečnost.

Preimage resistance

Vlastnost hashe: na hodnotu h je obtížné najít v takové, že $H(v) = h$. Pro SHA-256: 2^{256} klasicky, 2^{128} post-Grover.

$\lambda_{\text{preimage}}$

Kumulativní preimage bariéra. Kolik bitů hashových omezení útočník musí překonat na jeden password vektor. V referenci v2: 120 bitů.

h_{AB}, h_{BA}, h_P

Tři délky hashových prefixů ($A \rightarrow B$, $B \rightarrow A$, password). V ref v2: 5, 2, 8 bytů. Součet $\times 8 = \lambda_{\text{preimage}} = 120$ bitů.

$\mathcal{V}_A, \mathcal{V}_B$

Privátní pooly vektorů Alice resp. Boba. V ref v2 každý 4096 vektorů po 32 bytech.

$\mathcal{S}$ (Stat)

Statistická distribuce sdílená mezi A, B (i útočníkem). Pro každou pozici i pravděpodobnostní vektor přes 256 hodnot bytu. V kole se aktualizuje z veřejného R_t .

R_t Veřejný náhodný vektor v kole t . Přenášen $B \rightarrow A$, vstup do Stat aktualizace.

p_{upd}

Pravděpodobnost, že daný vektor v daném kole se aktualizuje. V ref v2: 0,75. Operační rozsah $[0,6, 0,95]$ vynucený.

M Počet password kandidátů na kolo. V ref v2: 8.

K^* Konečný sdílený klíč, výsledek protokolu. 32 bytů, použitelný jako AES-256 klíč.

Tail selekce / CandMax filter

Bob vybírá kandidáty s *nejmenší* pravděpodobností pod Stat (ne typické). Strukturální obrana proti přímému samplingu útočníkem.

NoPadding (PaddingMode.None)

AES režim bez paddingu. Klíč. property: dešifrování špatným klíčem produkuje pseudonáhodné bytes (ne explicitní chybu) $\Rightarrow$ žádné validační orákulum pro útočníka.

Stage-a / stage-b / stage-c

Postupné fáze útočnické filtrace. Stage-a: shodují se h_{AB} . Stage-b: shodují se $h_{AB} + h_{BA}$. Stage-c: shodují se $h_{AB} + h_{BA} + h_P$.

FH (Final-Hit)

Binární metrika: dosáhl protokol Final fáze (terminace)?

KM (Keys-Match)

Binární metrika: shodují se K^* Alice a Boba?

Clopper-Pearson CI

Standardní statistický horní/dolní interval na binomický odhad rate (počet úspěchů / n). Konzervativní, vhodný pro nízké rate (např. 0/300).

Cohenovo d

Standardní efekt-size metrika: $d = (\mu_1 - \mu_2)/\sigma$. $d = 0,2$ malý efekt, $d = 0,5$ střední, $d = 0,8$ velký. $d = 0,009$ je v Poisson šumu.

Konjekturálně

Pochází z latinského *coniectura*. V kryptografickém kontextu označuje tvrzení, které je strukturálně přesvědčivě argumentované (a často podpořené empirickou evidencí), ale dosud formálně matematicky nedokázané. Ne „domněnka“ v hovorovém smyslu — v matematice je konjektura precizně formulovaný předpoklad, jehož platnost je otevřená.

Reference v2

Aktuální produkční baseline parametrů (po pivotu 2026-05-02). $L = 32$, $N = 4096$, $h_{AB} = 5$, $h_{BA} = 2$, $h_P = 8$, $M = 8$, $p_{\text{upd}} = 0,75$, $\lambda = 120$ bitů.

reference_h_AB6

Kandidát na v3 reference. Klon reference v2 s $h_{AB} = 6$, $\lambda = 128$ bitů, +4 KB objemu dat na kolo.

break_demo

Záměrně oslabená research-only konfigur. $L = 8$, $\lambda = 56$ bitů — parametry těsně nad prahem, kde by útok teoreticky prošel. Pro pedagogickou demonstraci útoků, NE pro produkci.

10.2 Kde dál

Pokud chcete plnou matematiku. Paper v9 main: `paper/v9/files/paper_v9_cz.tex` (CZ) a `paper_v9_en.tex` (EN). Plné důkazy, formální definice, kompletní tabulky empirických výsledků, bibliography. Cca 1180 řádků LaTeXu, ~ 30 stran PDF.

Pokud chcete reprodukovat empiriku. `docs/REPRODUCE_PAPER.md`: step-by-step instrukce pro reprodukci hlavních empirických tvrzení. Tři úrovně:

- Lv0 sanity (~ 30 s): jednoduchý běh, ověření že implementace funguje.
- Lv1 small (~ 10 min): mini-konvergence + NIST malá baterie.
- Lv2-3 ($\sim 1\text{--}24$ h): plná validace 50k běhů + NIST 1000 klíčů + 5 tříd útočníků na 300 transkriptů.

Pokud chcete kód. GitHub: `GrataLuma/Cascade`. Implementace v C# (.NET 9). Reference v2 v `configs/reference.json`. Hlavní crypto v `GrataCascade.Core/`. CLI v `src/Program.cs`. Reporting v `Reports/`.

Pokud chcete kompletní glosář. `docs/glossary.md` (289 řádků): symboly, parametry, F-task index (F1–F13), profily, statistické konvence, konfigurace.

Pokud chcete přispět. Otevřené problémy z Kapitoly 9, prioritizované přibližně:

- **CKA rozšíření** (`prob:cka-extension`) — nejdůležitější follow-up směr; konkrétní research program ve třech cestách (A: repeated-session chaining, B: symetrický ratchet nad GC, C: nativní CKA).

- **Formální redukce** (`prob:dynamic-formal`) — second central problem; báze pro Path A v CKA rozšíření.
- **Flooding stress test** (`prob:flood-stress`) — empirická validace AES NoPadding obrany při $P = 10^{12}$, GPU-friendly (8–16 hodin na RTX 4060).
- **ML útočníci** (`prob:ml`) — nová třída, otevřený prostor.
- **Optimalizace objemu dat** (`prob:bandwidth`) — inženýrská, ale s velkým dopadem.
- **Per-profile útočnická evaluace** (`prob:parametric`) — mechanická, ale nutná.

Děkujeme, že jste dočetli. Pokud cokoli z této knížky bylo nejasné, nebo se vám zdá tvrzení sporné, otevřete prosím issue v repu — chceme, aby populární verze byla skutečně srozumitelná.

Problém narozenin popisuje statický stav. Grata Cascade na něm staví dynamický jednorázový systém. Rozšíření na continuous (CKA) variant je otevřené. A tahle dynamika, jak doufáme, vás přesvědčila.